UNIVERSITY OF AZAD JAMMU AND KASHMIR

DEPARTMENT OF SOFTWARE ENGINEERING

Course Title: Software Design & Architecture (Lab)

Course Code: SE-2204

Semester: Fall 2023

Contents

Lab-01: SRS Document 3
Lab-02: UML Disgram 9
Lab-03: Activity Diagram 16
Lab-04: Sequence Diagram 20
Lab-05: Class Diagram 25
Lab-06: Installation of Balsamiq 30
Lab-07: Mockup Design 34
Lab-08: Coupling concept by code analysis 37
Lab-09: Cohesion concept by code analysis 47
Lab-10: Data Flow Diagram 58
Lab-11: Design Principle 1 61
Lab-12: Design Principle II 70
Lab-13: Design Patterns I 77
Lab-14: Design Pattern II 89
Lab-15: Design Pattern III 92

Lab-01: SRS Document

1. Introduction

1.1 Purpose

The purpose of this document is to provide a detailed software requirements specification (SRS) for connecting a Raspberry Pi to an LED. The purpose of using a Raspberry Pi device with a LED is to create a versatile, portable computing platform that can be used for a wide range of applications, from education to home automation and beyond. By combining the power and flexibility of a Raspberry Pi with the convenience and mobility of a LED, users can create a computing system that is both powerful and easy to use.

1.2 Scope

The software to be developed will allow a Raspberry Pi to connect to an LED and control its on/off state.

Home Automation: Using a Raspberry Pi with a LED, users can control the on/off status of various smart home devices, such as lights, appliances, and thermostats. This can be done through a mobile app or a custom-built interface, allowing users to control their home environment with ease.

Industrial Automation: In industrial settings, a Raspberry Pi with a LED can be used to control the on/off status of various machines and equipment. This can be done through a custom-built interface that displays the status of each device and allows for easy control.

Security Systems: Using a Raspberry Pi with a tablet, users can control the on/off status of various security devices, such as cameras, alarms, and door locks. This can be done through a mobile app or a custom-built interface, allowing users to monitor and control their security system from anywhere.

1.3 Definitions, Acronyms, and Abbreviations LED

LED stands for Light Emitting Diode, which is a semiconductor device that emits light when an electric current passes through it. LEDs are commonly used in electronic devices and lighting applications due to their energy efficiency and long lifespan.

Raspberry Pi:

Raspberry Pi is a series of small single-board computers developed in the United Kingdom by the Raspberry Pi Foundation. These computers are designed for educational and hobbyist use and have gained popularity due to their low cost and versatility. Raspberry Pi boards can be used for a variety of projects, such as home automation, media centers, and robotics.

1.4 References

"Raspberry Pi and Android LED Integration," by Rajat Sharma. In this article, the author describes how to integrate a Raspberry Pi with LED using VNC (Virtual Network Computing) and SSH (Secure Shell) protocols. The article provides step-by-step instructions and tips for troubleshooting.

"Control Your Raspberry Pi from Your LED," by Thorin Klosowski. This article provides a detailed guide on how to use an LED to control a Raspberry Pi remotely. The author describes how to set up VNC and SSH, as well as how to use a mobile app called JuiceSSH.

"Raspberry Pi Projects for the Smart Home," by Jonathan Oxer and Hugh Blemings. This book provides a comprehensive guide to using a Raspberry Pi for home automation projects, including integrating with a LED. The book includes detailed instructions and diagrams for various projects, such as controlling lights and thermostats.

"Raspberry Pi for Secret Agents," by Stefan Sjogelid. This book provides a fun and engaging guide to using a Raspberry Pi for espionage-themed projects, including using a LED for remote surveillance. The book includes step-by-step instructions and code examples.

"Raspberry Pi and Tablets: A Perfect Match?" by James Bruce. In this article, the author explores the potential of using a Raspberry Pi with a tablet for various projects, such as home automation and gaming. The article provides an overview of the benefits and limitations of using a LED with a Raspberry Pi, as well as some project ideas.

2. Overall Description

2.1 Product Perspective

The software to be developed for this project will be a standalone application that will run on a Raspberry Pi. The Raspberry Pi is a small, low-cost computer that can be used for a variety of projects. In this case, it will be used to control the on/off state of an LED using its GPIO (General Purpose Input/Output) pins.

The application will be responsible for communicating with the GPIO pins on the Raspberry Pi and sending commands to turn the LED on or off. It will be designed to be simple and user-friendly, with a straightforward interface that allows the user to easily control the LED.

Since the software will be developed for a specific hardware platform (the Raspberry Pi), it will be important to ensure that it is compatible with the hardware and that it uses the GPIO pins correctly. The software will also need to be designed with security in mind, to prevent unauthorized access to the Raspberry Pi or the LED. Overall, the software will provide a simple and effective solution for controlling an LED using a Raspberry Pi.

2.2 Product Features

The software will have the following features:

2.2.1 Control LED On/Off State

The software will allow the user to control the on/off state of an LED connected to the Raspberry Pi. The user will be able to turn the LED on or off using a simple interface on their device.

2.2.2 LED Brightness Control

The software will also allow the user to control the brightness of an LED connected to the Raspberry Pi. The user will be able to adjust the brightness level using the interface provided by the software.

2.2.3 User Interface

The software will have a user interface that allows the user to control the LED. The interface will be simple and intuitive, with clear instructions on how to use it. The user will be able to turn the LED on or off, adjust the brightness level, and perform other related functions.

2.3 User Characteristics

The intended users of this software are people who have basic knowledge of electronics and programming. The software is designed to be easy to use, with clear instructions provided to guide the user through the process. However, it is assumed that the user has a basic understanding of electronics and programming, in order to ensure that they can use the software effectively.

3. Specific Requirements

3.1 Functionality

The software will meet the following functional requirements:

3.1.1 Control LED On/Off State

The software shall allow the user to control the on/off state of an LED connected to the Raspberry Pi. The user will be able to turn the LED on or off using the software interface.

3.1.2 LED Brightness Control

The software shall allow the user to control the brightness of an LED connected to the Raspberry Pi. The user will be able to adjust the brightness level using the software interface.

3.1.3 User Interface

The software shall have a user interface that allows the user to control the LED. The interface will be designed to be simple and intuitive, allowing the user to easily navigate and control the LED.

3.2 Performance Requirements

The software shall be able to control the LED with minimal latency and delay. This means that the software will be designed to operate quickly and efficiently, with minimal delay between the user's actions and the LED response.

3.3 Design Constraints

The software must be designed to work with the Raspberry Pi and its GPIO pins. This means that the software will be designed to interact with the hardware of the Raspberry Pi, specifically its GPIO pins, to control the LED.

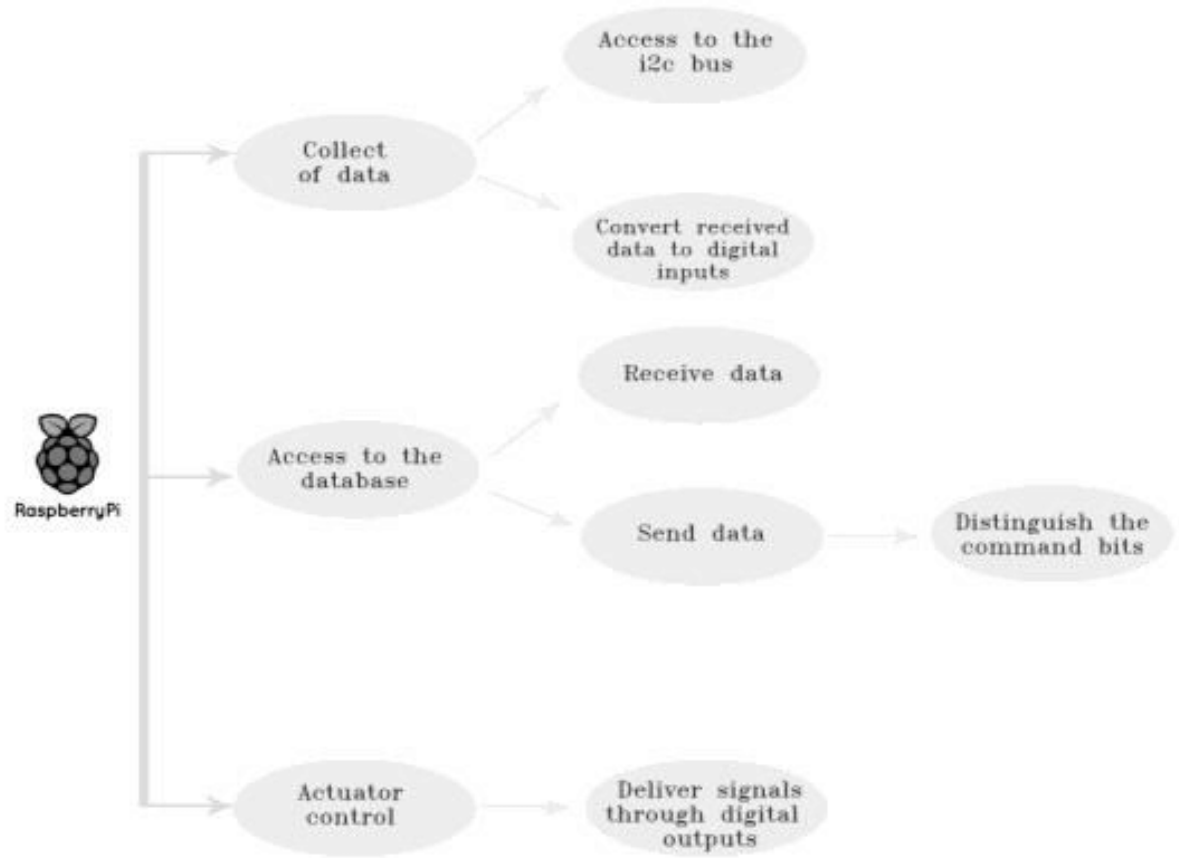
3.4 Assumptions and Dependencies

The software assumes that the Raspberry Pi is connected to the LED and has the necessary libraries and software installed to interface with the GPIO pins. The software also assumes that the user has basic knowledge of electronics and programming, as discussed earlier.

4. System Models

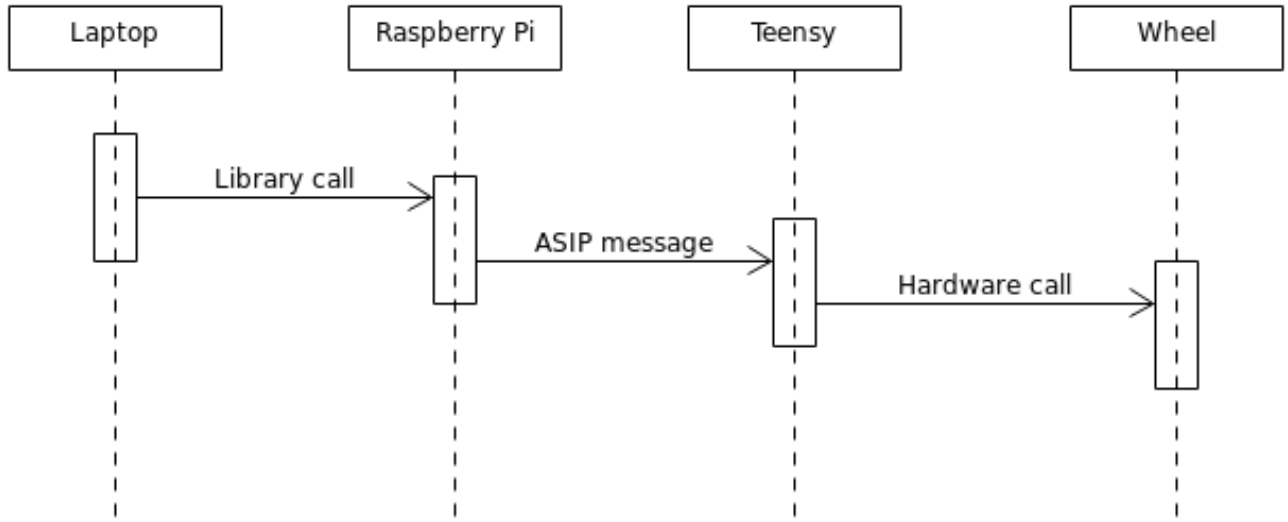
4.1 Use Case Model

The use case diagram for the software is as follows:



4.2 Sequence Diagram

The sequence diagram for controlling the LED is as follows:



Conclusion

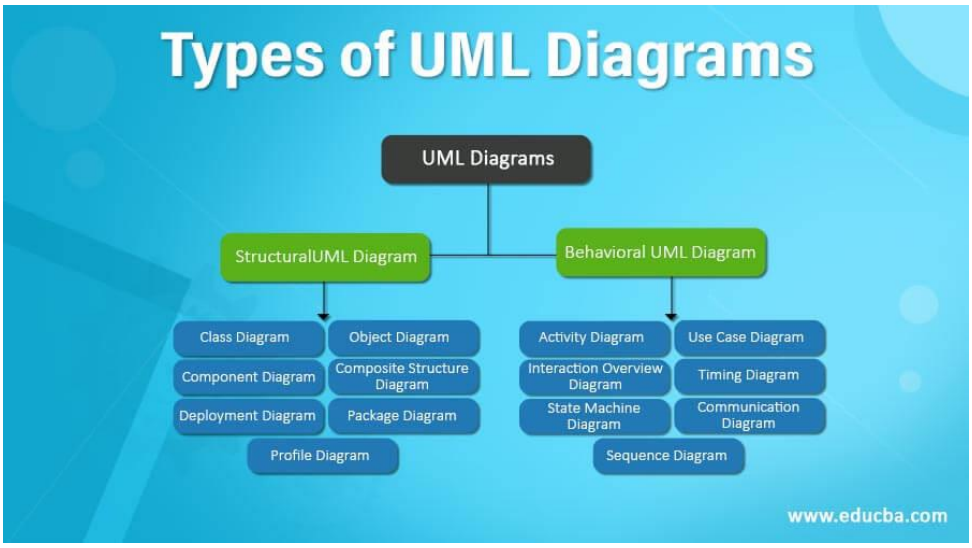
This SRS document has outlined the requirements for software that will allow a Raspberry Pi to connect to an LED and control its on/off state. The software will have the ability to control the brightness of the LED and will have a user interface to allow the user to interact with the system. The software shall be able to control the LED with minimal latency and delay. The use case and sequence diagrams have been provided to illustrate the functionality and flow of the software.

Lab-02: UML Disgram

1 UML diagrams

A UML diagram is a visual representation of a system using the Unified Modeling Language (UML). UML is a standard modeling language used in software engineering to depict the various aspects of a system. The following are some of the most common types of UML diagrams:

Types



Class Diagram: A class diagram represents the static structure of a system by showing classes, their attributes, and their relationships.

Use Case Diagram: A use case diagram describes the functionality provided by a system in terms of actors, their goals, and the use cases that achieve those goals.

Sequence Diagram: A sequence diagram depicts interactions between objects in a system over time.

Activity Diagram: An activity diagram illustrates the flow of activities within a system.

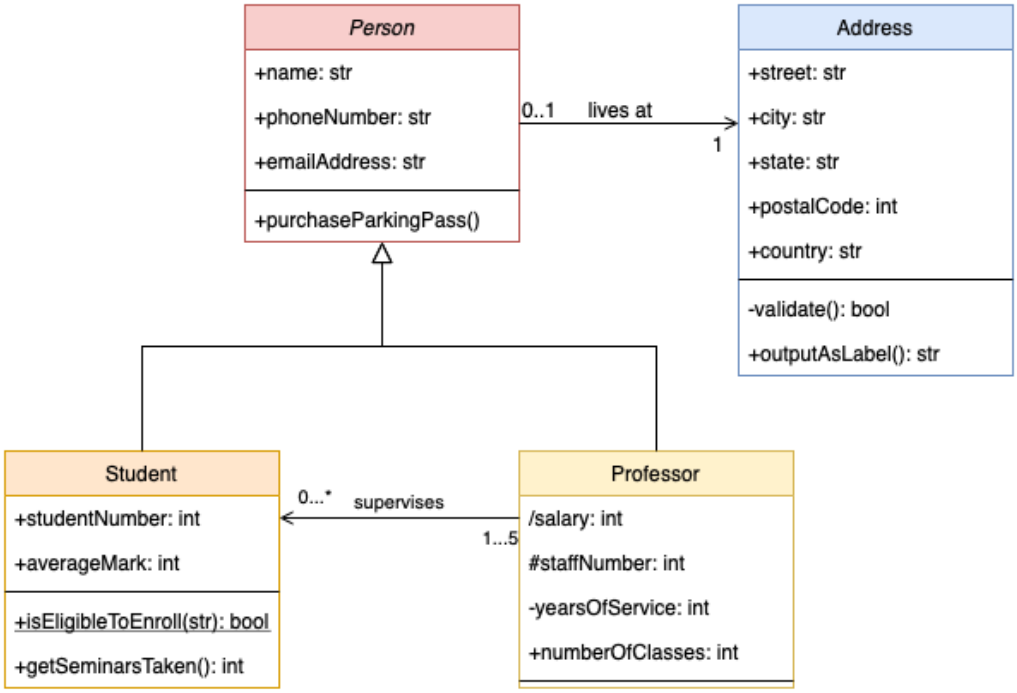
State Machine Diagram: A state machine diagram depicts the behavior of a system or object in response to events and actions.

Communication Diagram: A communication diagram is a simplified version of a sequence diagram that shows the interactions between objects or actors in a system.

Component Diagram: A component diagram illustrates the components of a system and their relationships.

Deployment Diagram: A deployment diagram shows the physical configuration of a system and how its components are deployed.

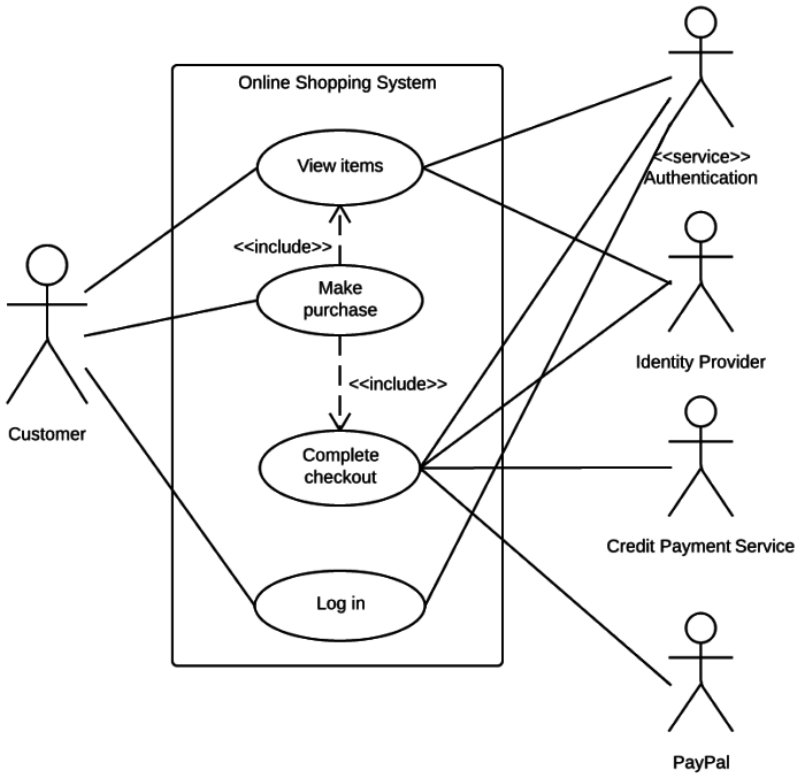
1.2 Diagram



Use case diagrams

A use case diagram is a type of UML (Unified Modeling Language) diagram that represents the interactions between the actors (users or systems) and the system under consideration to achieve a specific goal. It depicts the functionality of a system from the user's perspective and is used to describe the behavior of a system in a concise and easily understandable way.

The relationships between actors and use cases can be represented by different types of connectors, such as association, generalization, and include/extend. Association represents a relationship between an actor and a use case, generalization represents an "is-a" relationship between actors or use cases, and include/extend represents the relationships between use cases that depend on other use cases.



Components

The main components of a use case diagram include:

Actors

Actors are the external entities that interact with the system. They represent the roles that users or other systems play in the system under consideration. Actors are represented by stick figures in a use case diagram.

Use Cases

Use cases represent the specific tasks or services that the system provides to its actors. They describe the system's behavior from the actor's perspective. Use cases are represented by ovals in a use case diagram.

Relationships

Relationships are the connections between actors and use cases. There are three types of relationships in a use case diagram:

Association

An association represents a relationship between an actor and a use case. It is shown by a solid line between the actor and the use case.

Generalization

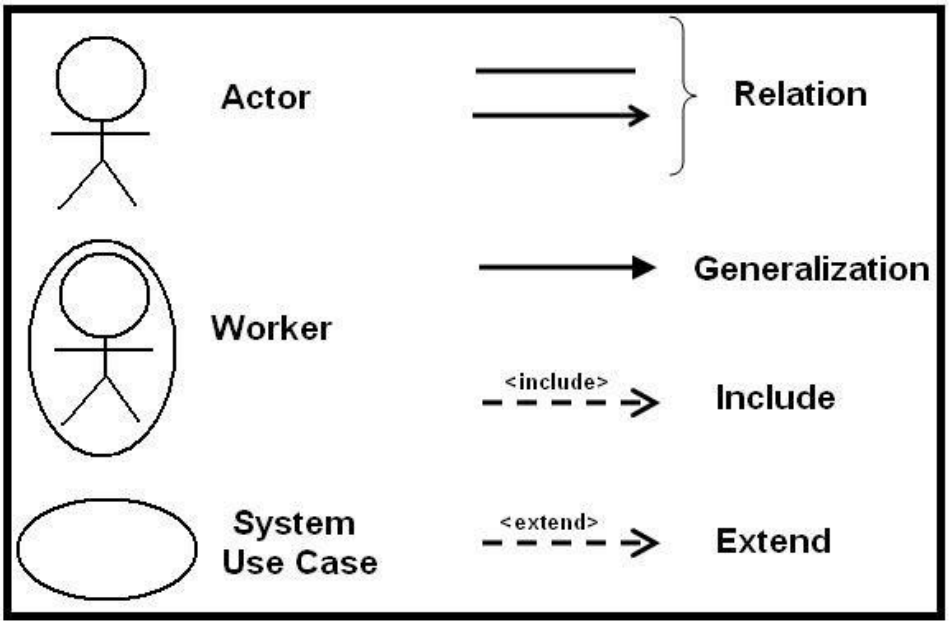
A generalization represents an "is-a" relationship between two actors or two use cases. It is shown by a solid line with a triangle pointing to the parent actor or use case.

Include/Extend

The include/extend relationships represent the dependencies between use cases. The include relationship indicates that one use case is always included in another use case. The extend relationship indicates that one use case can optionally extend another use case. These relationships are shown by a dotted line with an arrowhead pointing to the included/extended use case.

System Boundary

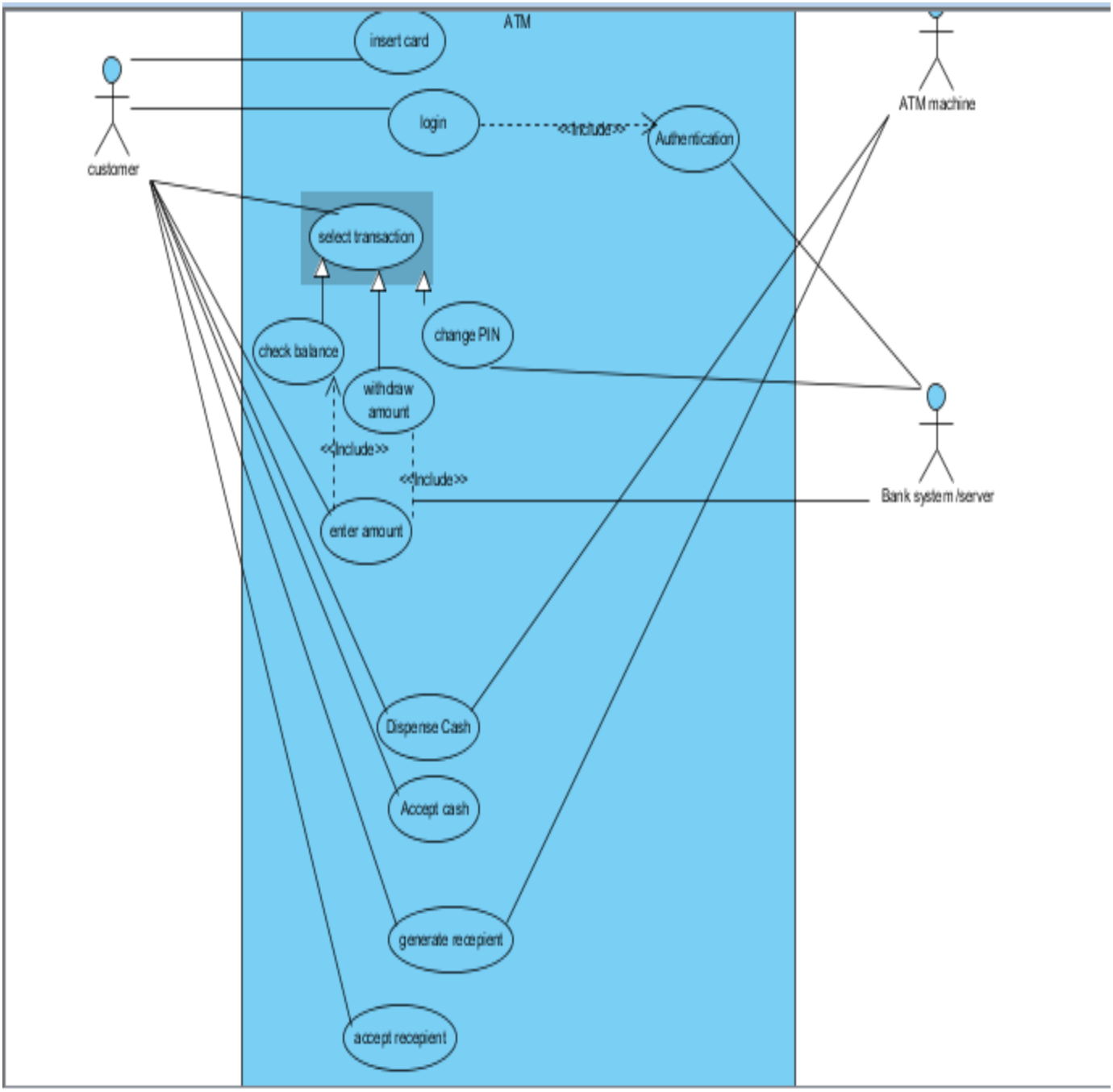
The system boundary represents the boundary of the system under consideration. It is used to indicate the scope of the use case diagram and the system's interactions with the external entities. The system boundary is represented by a rectangle enclosing the actors and use cases.



2 IMPLEMENTATIONS

2.1 ATM use case

Q 1: Using your knowledge of how an Automated Teller Machine (ATM) is used, draw a use case diagram for it.



2.2 Online cake orders

Q 2: Create the UML Use Case Diagram for a system to buy cake online. Customers, Delivery Agent, and Administrators of the System will use the system. They can perform the following basic actions in the system:

Customer:

Add a cake to shopping cart.

Remove a cake from shopping cart.

Check out shopping cart after placing the order and payment.

Search cake by flavors.

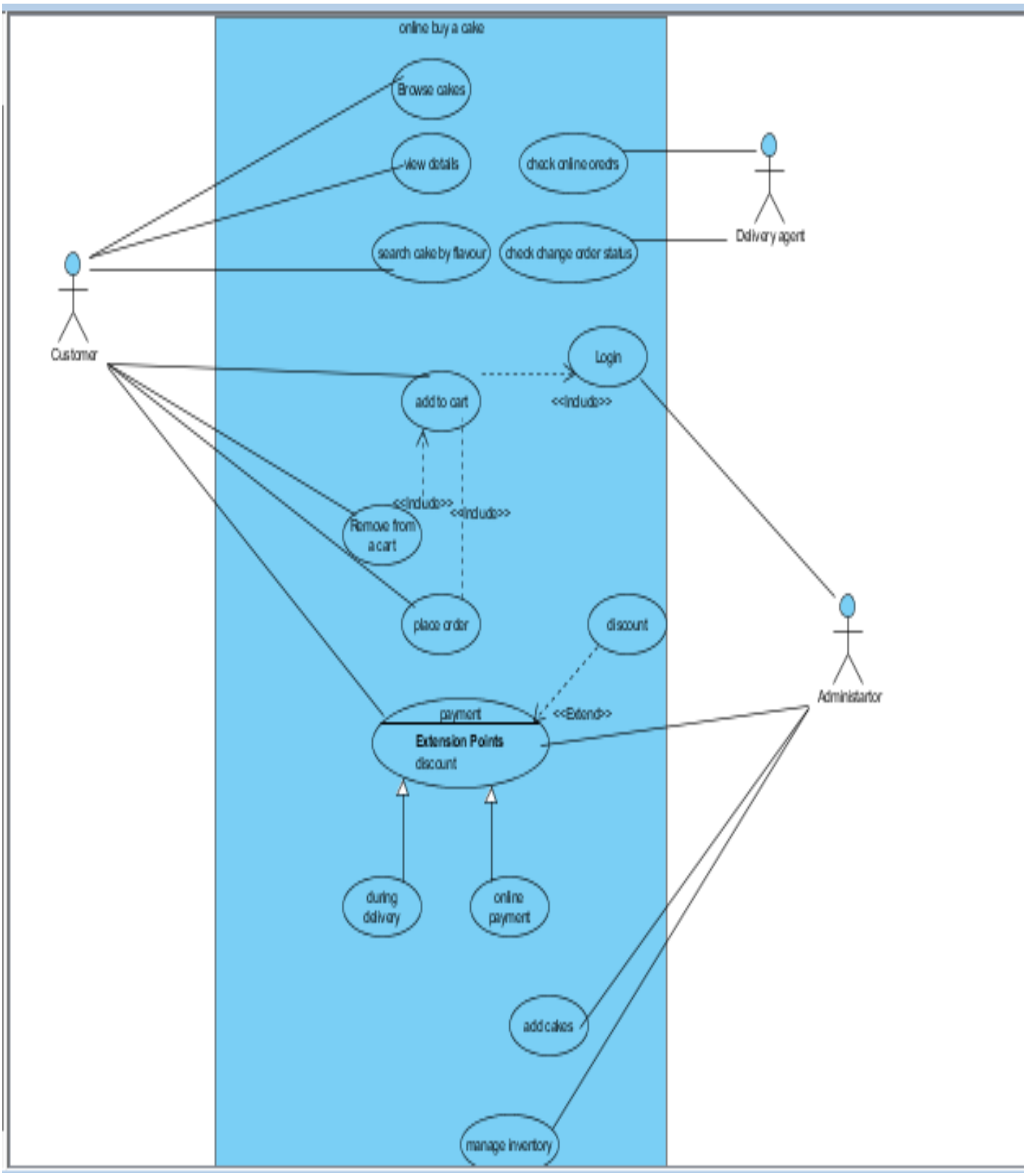
Delivery Agent:

Check online orders

Change the status of the order to deliver Administrator:

Add cakes

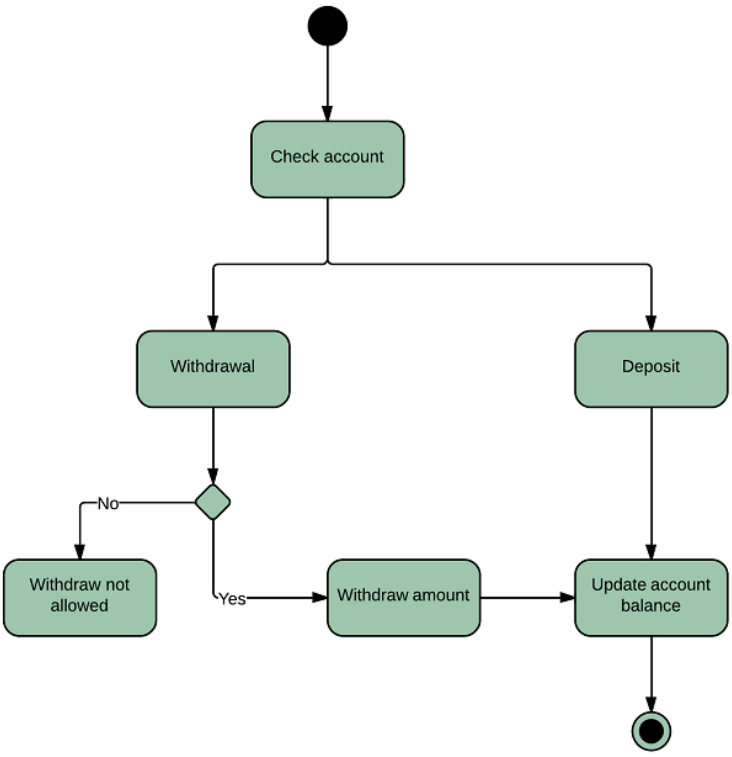
Update inventory of cakes


online buy a cake
Browse cakes
view details
check online credts
Delivery agent
search cake by flavour
check change order status
Customer
Login
add to cart
<<Include>>
<<Include>>
<<Include>>
Remove from a cart
place order
discount
Administartor
payment
<<Extend>>
Extension Points
discount
during delivery
online payment
add cakes
manage inventory

Lab-03: Activity Diagram

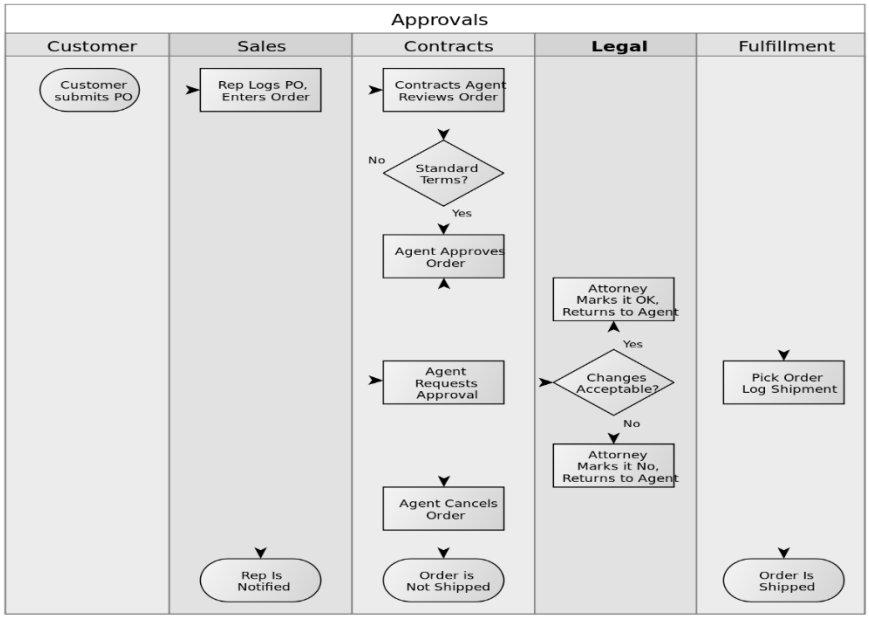
Activity diagram

An activity diagram visually presents a series of actions or flow of control in a system similar to a flowchart or a data flow diagram. Activity diagrams are often used in business process modeling. They can also describe the steps in a use case diagram. Activities modeled can be sequential and concurrent.



Swim lane diagram

A swim lane diagram is a type of flowchart that delineates who does what in a process. Using the metaphor of lanes in a pool, a swim lane diagram provides clarity and accountability by placing process steps within the horizontal or vertical “swim lanes” of a particular employee, work group or department.



1.4 Components

A swim lane map is similar to a flowchart in that it maps out a process, decisions, and loops; however, a swim lane map places events and actions in "lanes" to delineate a person/group responsible, or a specific sub process. A swim lane map has three elements: time, people (or job functions), and tasks/processes.

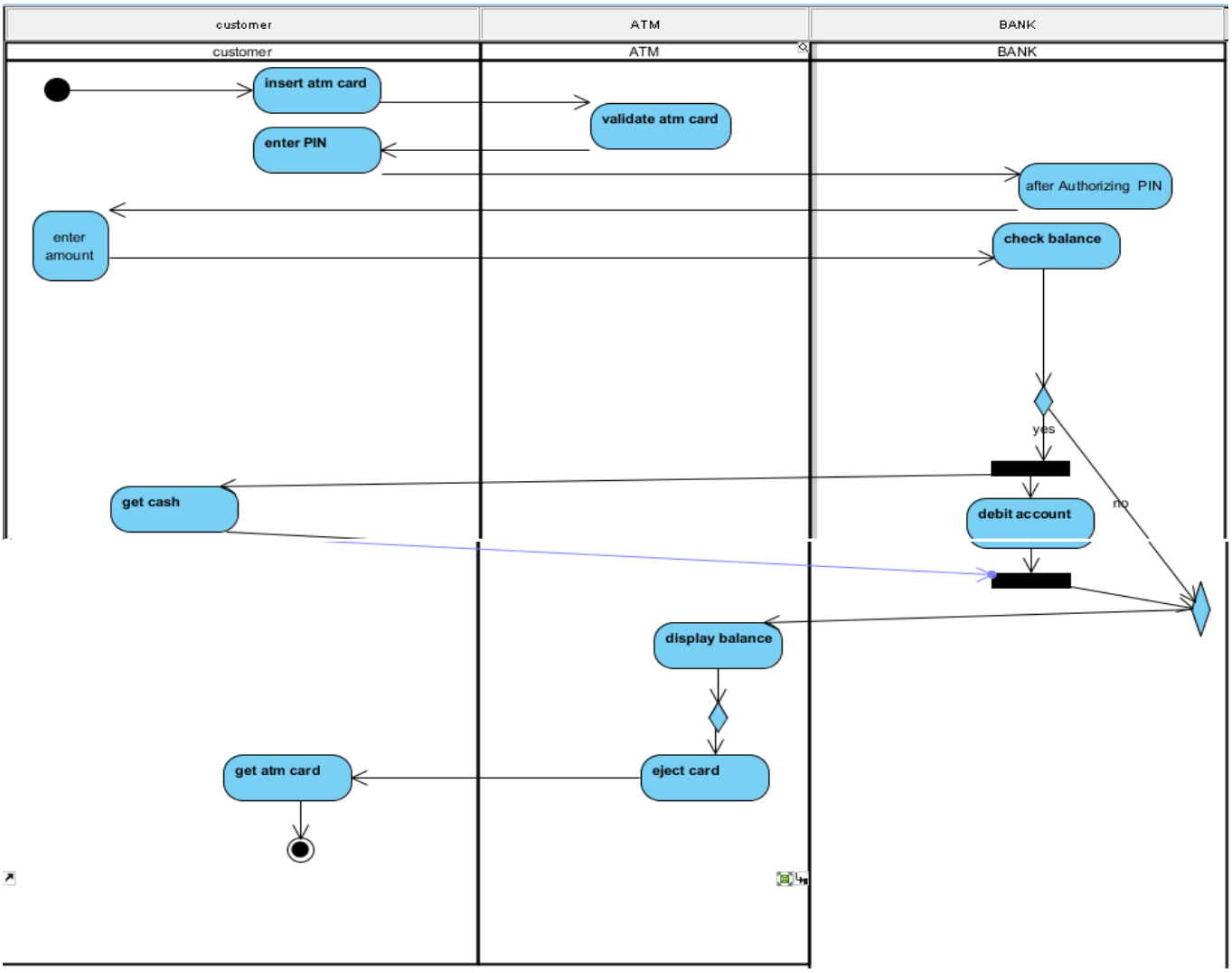
- **Starting point**
- **Ending point**
- **Rounded rectangle**
- **Flow lines**
- **Decision symbol**
- **Fork**
- **Join**

1.5 Features

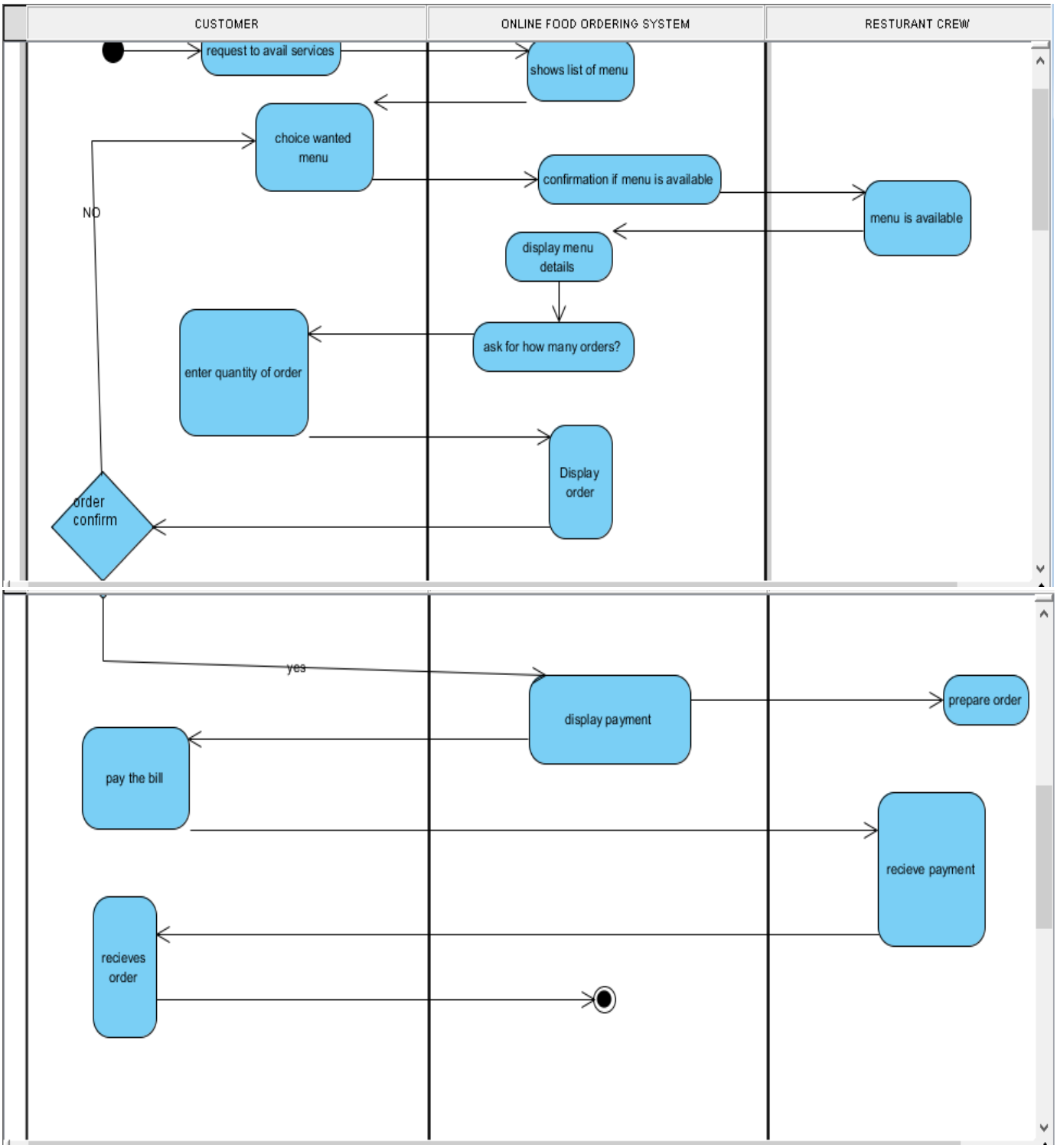
Swim lane diagrams are flowcharts that show a process from start to finish. These diagrams also show who is responsible for each step in the process. Much like a swimming pool with established lanes for each swimmer, a swim lane diagram has horizontal or vertical lanes belonging to each person involved in the process.

2 IMPLEMENTATIONS

2.1 ATM swim lane



2.2 Food panda

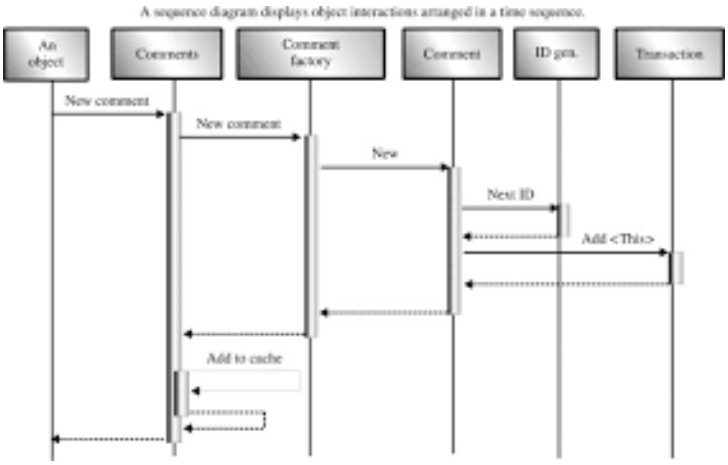

CUSTOMER
ONLINE FOOD ORDERING SYSTEM
RESTURANT CREW
request to avail services
shows list of menu
choice wanted menu
confirmation if menu is available
menu is available
NO
display menu details
ask for how many orders?
enter quantity of order
Display order
order confirm
yes
display payment
prepare order
pay the bill
recieve payment
recieves order

Lab-04: Sequence Diagram

1 Sequence diagram

1.1 Definition

A sequence diagram is a type of UML (Unified Modeling Language) diagram that represents the interactions between objects in a system. It shows the flow of messages between objects and the order in which those messages are sent and received. Sequence diagrams are particularly useful for modeling complex systems with multiple components or objects that interact with one another.



1.2 Components of a Sequence Diagram:

A sequence diagram consists of several key components:

Objects

Objects are represented as rectangles with the name of the object inside. Each object is typically associated with a class in the system.

Lifelines

Lifelines are vertical dashed lines that represent the lifespan of an object in the system. They extend from the top of the object's rectangle to the bottom of the diagram.

Messages

Messages are represented as arrows that move between lifelines. They show the flow of communication between objects. Messages can be synchronous (blocking) or asynchronous (non-blocking).

Activation Bars

Activation bars are horizontal bars that represent the time during which an object is actively processing a message. They are drawn on the lifeline of the object that is processing the message.

Guards

Guards are conditions that must be met in order for a message to be sent. They are typically represented as square brackets around the message.

Iterations

Iterations are represented by a loop symbol around a message. They show that a message is being sent repeatedly.

Combined Fragments

Combined fragments are used to show complex interactions between objects. They can be used to represent conditions, loops, or alternative paths.

1.3 Uses of Sequence Diagram

Requirement analysis: Sequence diagrams can be used to analyze and document system requirements by illustrating the interactions between objects in the system.

Design

Sequence diagrams are useful for designing software systems by breaking down complex interactions into smaller, more manageable parts.

Communication

Sequence diagrams can be used to communicate system requirements and design to stakeholders, including developers, testers, and project managers.

Testing

Sequence diagrams can be used to design and execute system tests by simulating interactions between objects in the system.

1.4 Example of a Sequence Diagram

Consider a simple example of a sequence diagram that shows the interaction between a customer and a cashier in a grocery store.

The customer selects items and places them on the conveyor belt.

The cashier scans the items and calculates the total cost.

The cashier requests payment from the customer.

The customer provides payment.

The cashier gives change (if applicable) and the customer collects their items.

Another example

1.5 Conclusion

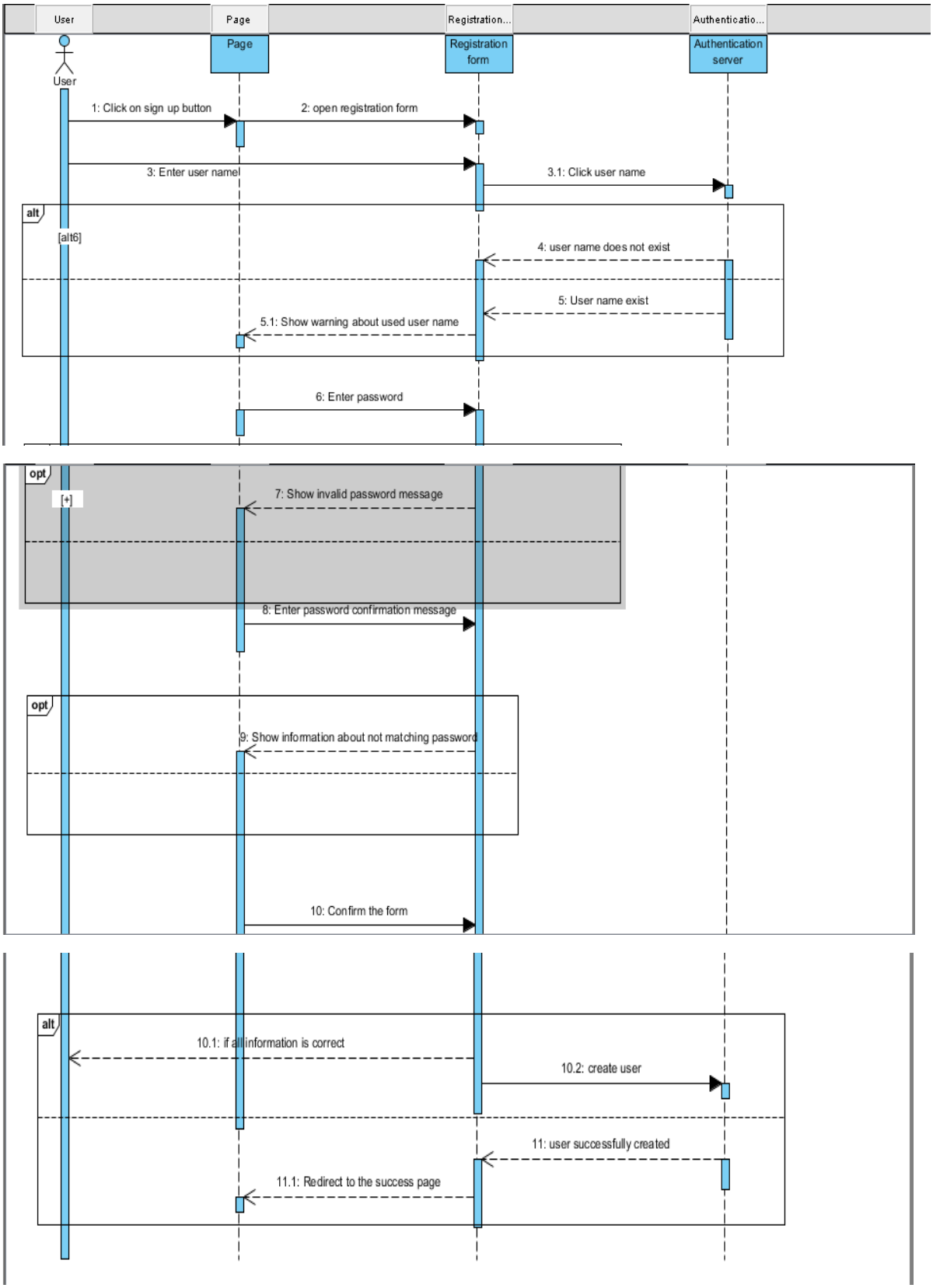
In summary, sequence diagrams are a powerful tool for modeling the interactions between objects in a system. They can be used for requirement analysis, design, communication, and testing. By breaking down complex interactions into smaller, more manageable parts, sequence diagrams help developers to build better software systems.

Bottom of Form

2 IMPLEMENATIONS

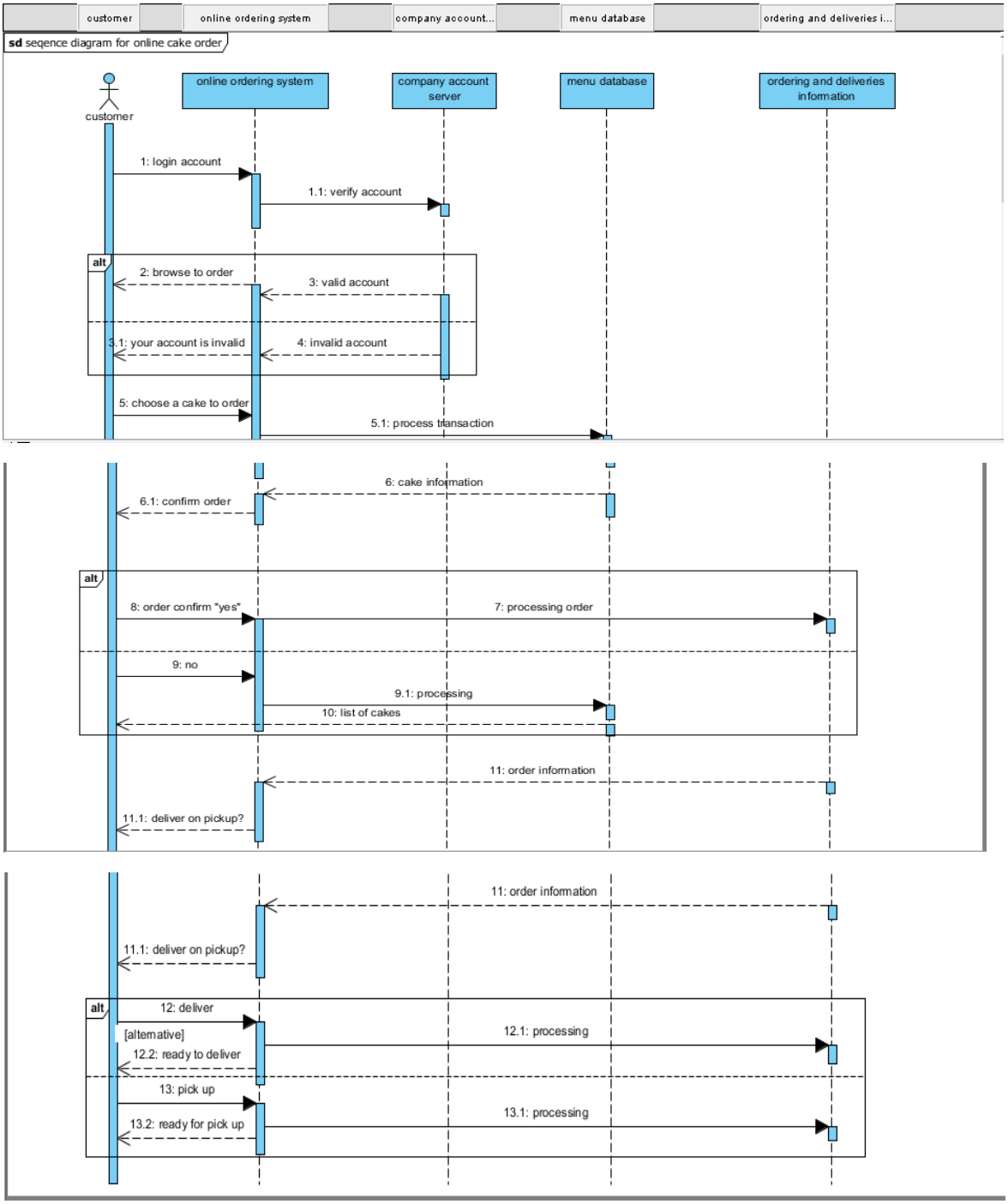
2.1 Sign up page

Q 1: Draw a sequence diagram for sign-up page.


User
Page
Registration...
Authenticatio...
Page
Registration form
Authentication server
User
1: Click on sign up button
2: open registration form
3: Enter user name
3.1: Click user name
alt
[alt6]
4: user name does not exist
5: User name exist
5.1: Show warning about used user name
6: Enter password
opt
[+]
7: Show invalid password message
8: Enter password confirmation message
opt
9: Show information about not matching password
10: Confirm the form
alt
10.1: if all information is correct
10.2: create user
11: user successfully created
11.1: Redirect to the success page

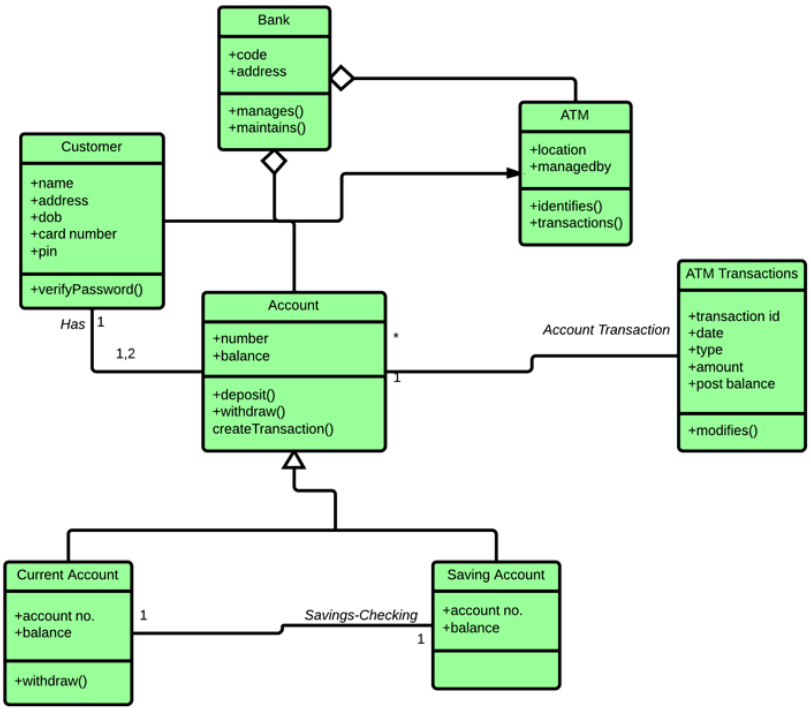
2.2 Online cake order

Q 2: Draw a sequence diagram for an online cake order system. The system should be based on the use case scenario provided in the previous task.



Lab-05: Class Diagram
1 Class diagram

A class diagram is a type of UML diagram that shows the structure of a system by modeling its classes, their attributes, methods, and relationships with other classes. A class diagram provides a high-level view of a system's architecture and helps in visualizing the overall design of a software application.



1.1 Basic Elements of a Class Diagram

Relationships in a Class Diagram a. Association b. Aggregation c. Composition d. Generalization e. Specialization

Basic Elements of a Class Diagram

A class diagram consists of several basic elements, including:

Class

A class is a template that defines a set of attributes, operations, and relationships that are common to a group of objects. A class is represented by a rectangle with three compartments: the top compartment contains the class name, the middle compartment contains the attributes, and the bottom compartment contains the methods.

Attribute

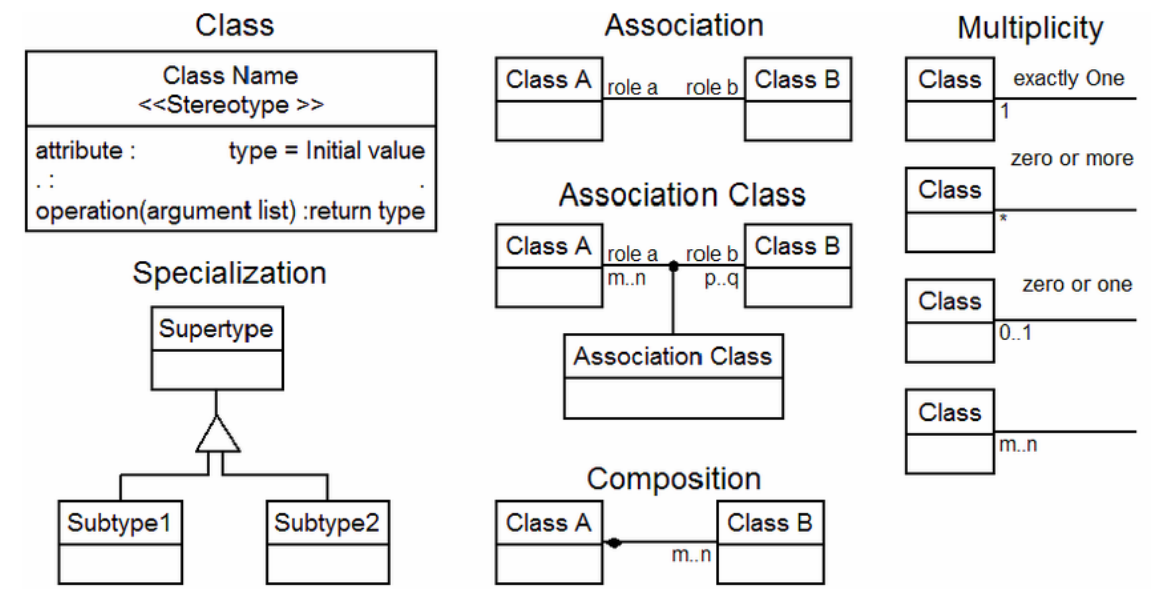
An attribute is a property or characteristic of a class that describes its state. An attribute is represented by a name-value pair that describes the data type and visibility of the attribute.

Method

A method is an operation or behavior of a class that can be invoked by other classes. A method is represented by a name, a list of parameters, and a return type.

Visibility

Visibility specifies the accessibility of a class, attribute, or method to other classes. There are three types of visibility: public (+), private (-), and protected (#).

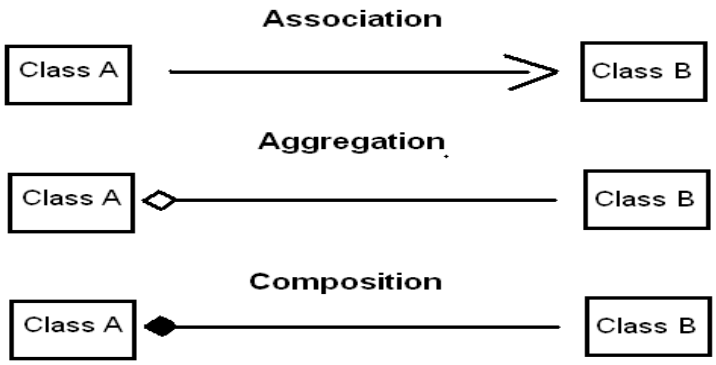


1.2 Relationships in a Class Diagram

Relationships in a class diagram describe how one class is related to another class. There are five types of relationships in a class diagram: association, aggregation, composition, generalization, and specialization.

a. Association

An association represents a relationship between two or more classes. It describes how objects of one class are connected to objects of another class. An association is represented by a line connecting the classes, and it may have an optional name, cardinality, and role.



b. Aggregation

Aggregation is a special type of association that represents a whole-part relationship between two classes. It describes how objects of one class are composed of objects of another class. Aggregation is represented by a diamond shape at the end of the line connecting the classes.

c. Composition

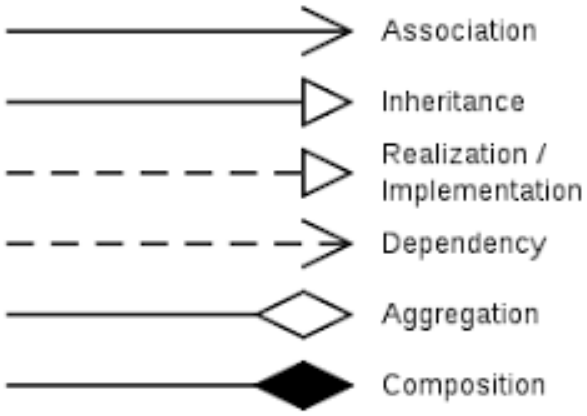
Composition is a stronger form of aggregation that represents a life cycle dependency between two classes. It describes how objects of one class are responsible for creating and destroying objects of another class. Composition is represented by a filled diamond shape at the end of the line connecting the classes.

d. Generalization

Generalization represents an "is-a" relationship between two classes. It describes how a subclass inherits the attributes and behaviors of a superclass. Generalization is represented by a solid line with a closed arrowhead pointing from the subclass to the superclass.

e. Specialization

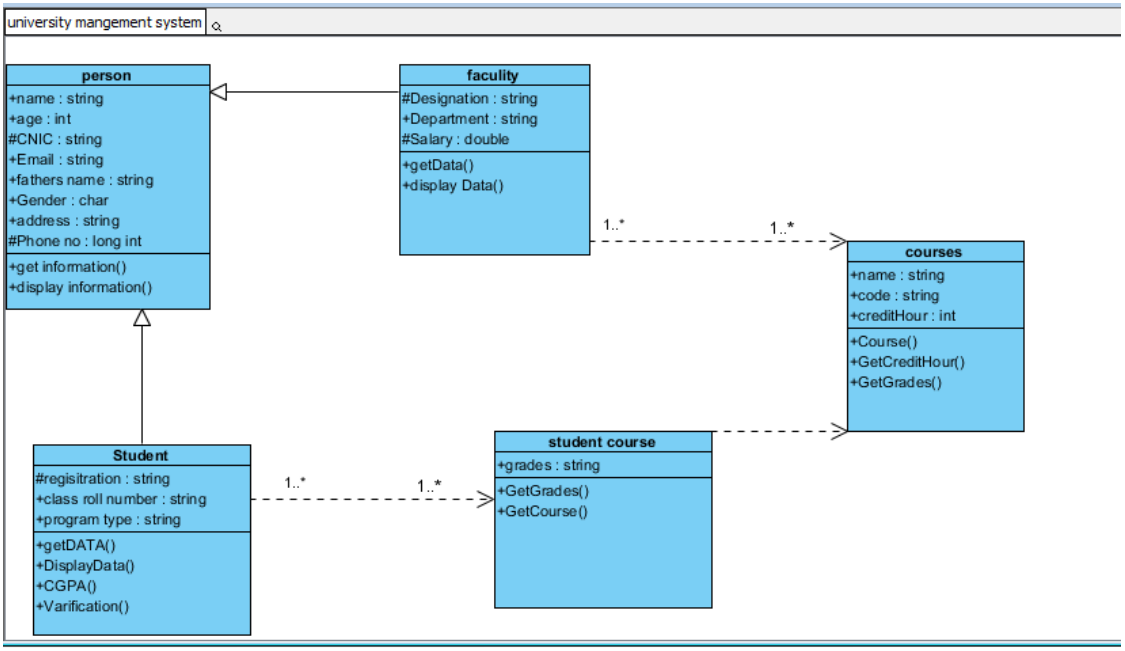
Specialization represents a type of generalization that describes how a subclass inherits only some of the attributes and behaviors of a superclass. It is represented by a solid line with an open arrowhead pointing from the subclass to the superclass.



2 IMPLEMENTATIONS

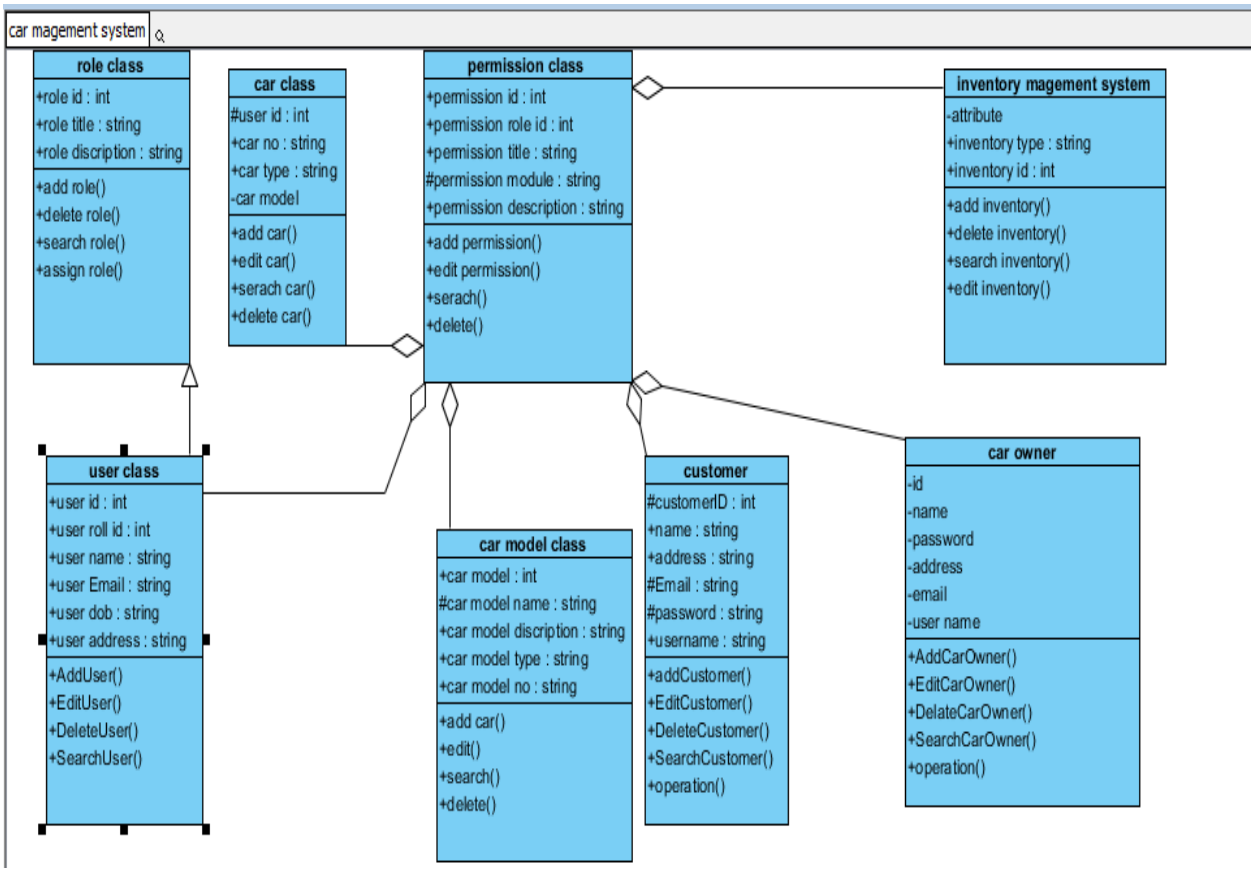
2.1 University Management System

Q 1: A university has a course management system that allows students to register for courses, and for professors to manage course content and grades. The system should be able to handle multiple courses and multiple students in each course.



2.2 Car management system

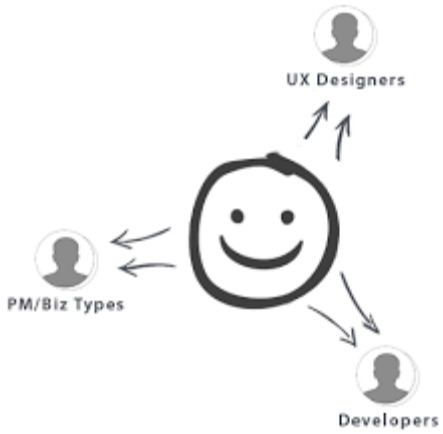
Q 2: A car dealership needs a management system to keep track of its inventory of cars, including new and used cars, as well as the employees who work at the dealership. The system should also allow customers to view the available cars and make purchases.


car magement system
role class
+role id : int
+role title : string
+role discription : string
+add role()
+delete role()
+search role()
+assign role()
car class
#user id : int
+car no : string
+car type : string
-car model
+add car()
+edit car()
+serach car()
+delete car()
permission class
+permission id : int
+permission role id : int
+permission title : string
#permission module : string
+permission description : string
+add permission()
+edit permission()
+serach()
+delete()
inventory magement system
-attribute
+inventory type : string
+inventory id : int
+add inventory()
+delete inventory()
+search inventory()
+edit inventory()
user class
+user id : int
+user roll id : int
+user name : string
+user Email : string
+user dob : string
+user address : string
+AddUser()
+EditUser()
+DeleteUser()
+SearchUser()
car model class
+car model : int
#car model name : string
+car model discription : string
+car model type : string
+car model no : string
+add car()
+edit()
+search()
+delete()
customer
#customerID : int
+name : string
+address : string
#Email : string
#password : string
+username : string
+addCustomer()
+EditCustomer()
+DeleteCustomer()
+SearchCustomer()
+operation()
car owner
-id
-name
-password
-address
-email
-user name
+AddCarOwner()
+EditCarOwner()
+DelateCarOwner()
+SearchCarOwner()
+operation()

Lab-06: Installation of Balsamiq

1 Balsamiq

Balsamiq is a popular software tool used for creating wireframes and prototypes for digital interfaces. It provides a user-friendly and intuitive platform for designers, developers, and product managers to quickly sketch out and visualize their ideas for websites, web applications, mobile apps, and other software projects.



Website prototyping on Balsamiq

Balsamiq is an excellent tool for website prototyping, offering a range of features and functionality that make it ideal for visualizing and refining website designs. Here's how you can use Balsamiq for website prototyping:

Plan the Website Structure:

Before diving into the design, it's crucial to outline the structure and organization of your website. Start by creating a wireframe for the homepage, and then sketch out wireframes for each page or section of your website. Balsamiq provides pre-built UI components like headers, footers, navigation menus, and content sections that you can drag and drop onto the canvas.

Design Page Layouts: With the overall structure in mind, you can start designing individual page layouts using Balsamiq's components. Consider the placement of elements like logos, navigation menus, content sections, images, and buttons. Balsamiq's simplicity and library of UI elements make it easy to experiment with different layouts and arrangements.

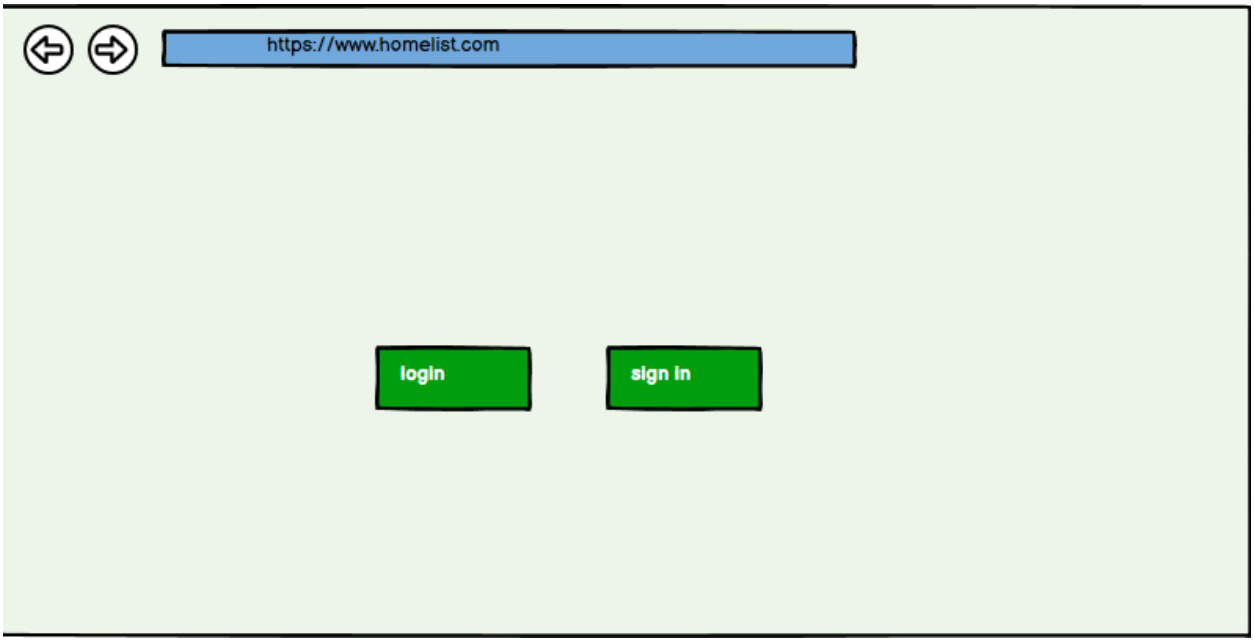
Visualize Interactions and Navigation: Use Balsamiq's interactive features to simulate user interactions and website navigation. You can create clickable areas on buttons, links, or menus to demonstrate how users would navigate between pages or interact with specific elements. This helps in refining the user flow and ensuring a seamless browsing experience.

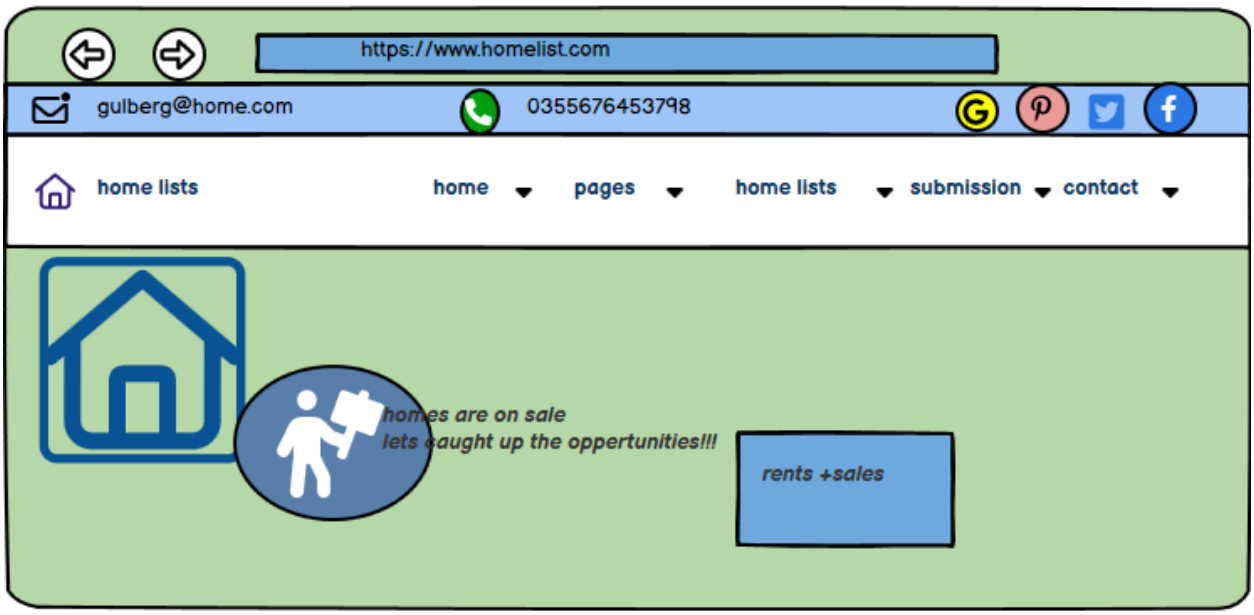
Incorporate Visual Design Elements: Although Balsamiq focuses on low-fidelity wireframes, you can add visual design elements to give an impression of the final website's aesthetics. For example, you can use Balsamiq's text styles, colors, and simple graphics to provide a sense of the overall visual direction. However, it's important to remember that Balsamiq is primarily used for conceptualizing and refining the layout and structure, rather than pixel-perfect visual design.

Gather Feedback and Iteration: Share your Balsamiq wireframes with stakeholders, clients, or potential users to gather feedback and insights. Balsamiq allows you to export your wireframes as PDFs or images, making it easy to share and discuss your designs. Based on the feedback received, iterate on your wireframes to refine the layout, information architecture, and overall user experience of the website.

Handoff to Web Development: Once the wireframes are finalized and approved, you can provide them to your web development team. Although Balsamiq does not generate code directly, the wireframes serve as a visual reference that developers can use to understand the intended layout and interactions. Developers can then translate the designs into HTML, CSS, and other web technologies using tools like code editors and web development frameworks.

IMPLEMENTATION




https://www.homelist.com
gulberg@home.com
0355676453798
home lists
home
pages
home lists
submission
contact
homes are on sale
lets caught up the oppertunities!!!
rents +sales

RENTS AND SALES HERE
RENTS AND SALES HERE
LOCATIONS
ALL STATUS
CONTACT
PRICE
ROOMS
here all rents ,sales are available for people ;
you may buy and sale here ! so what are you
waiting for

Lab-07: Mockup Design

1.2 Android application on Balsamiq

Balsamiq for Android application design:

Start with Sketching: Balsamiq's sketch-style interface is ideal for the initial ideation phase. Begin by sketching out rough ideas for your app's screens and interactions using the pre-built UI components provided by Balsamiq. These components include buttons, text fields, navigation bars, and other common Android UI elements.

Create Screen Mockups: Use Balsamiq's drag-and-drop functionality to assemble your app's screen layouts. Focus on the arrangement of UI elements, such as buttons, text, images, and navigation menus, without worrying too much about the visual design details at this stage. Balsamiq's simplicity allows you to quickly iterate and experiment with different layout options.

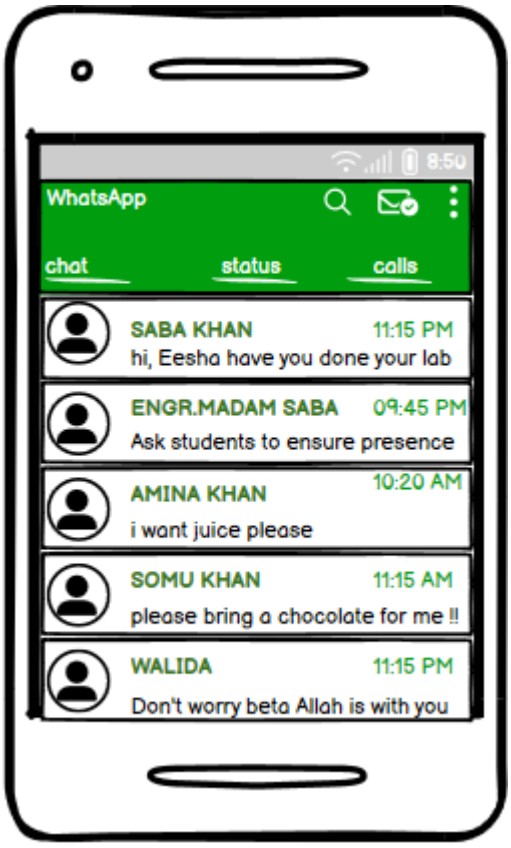
Design Navigation Flows: Define the navigation flow of your Android app by linking the screens together using Balsamiq's interactive features. You can create clickable areas on buttons or menus to simulate how the user would navigate between different screens or perform specific actions. This helps to test and refine the app's user flow and information architecture.

Incorporate Android-Specific Elements: While Balsamiq provides a generic set of UI components, you can customize and adapt them to match Android's design guidelines. Add specific Android elements like tabs, navigation drawers, and floating action buttons to ensure your app follows the platform's standards and conventions.

Gather Feedback and Iterate: Share your Balsamiq mockups with stakeholders, clients, or potential users to gather feedback and insights. Balsamiq allows you to export your wireframes as PDFs or images, making it easy to share and discuss your designs. Based on the feedback

received, iterate on your wireframes to refine the UI/UX and address any concerns or suggestions.

2 IMPLEMENTATION



8:50
SABA KHAN
what are you doing saba?
i am doing well!!
can you share idea on prototyping with me
yes, of course why not!!

Lab-08: Coupling concept by code analysis

Code

```
#include <iostream>
using namespace std;

template <class A, class B>
class Calculator
{
    A num1;
    B num2;

public:
    Calculator(A num1, B num2)
    {
this->num1 = num1;
this->num2 = num2;
    }
A addition()
    {
 return num1 + num2;
    }

 A subtraction()
    {
 return num1 - num2;
    }
A multiplication()
```

```
    {    return num1 * num2;
    }

  A division()
    {
   return num1 / num2;
    }
};
void line()
{
            cout<<endl<<string(120,'_')<<endl;
}

int main()
{
    int option;
    char another,terminate;
    program:
    cout<<"\n\n\tenter choice "<<endl;
    cout << "\n\t 1 for Integers Calculation \n\t 2 for  Floating Calculations 3 for Integer &
Floating" << endl;
    cout<<"\t";
    cin >> option;
    switch (option)
    {
    case 1:
    {
int choice;
int a, b;
```

```
cout << "\tEnter two integers ";
cout<<"\t";
cin >> a >> b;
Calculator<int, int> cal(a, b);
again1:
cout<<"\n\n\tEnter your Choice"<<endl;
cout << "\n\t1) Addition \n\t2) Subtraction \n\t3) Multiplication \n\t4) Division" << endl;
cout<<"\t";
cin >> choice;
system("cls");
line();
switch (choice)
{
case 1:
cout << "\tAddition :"<<a<<" & " <<b <<" is " << cal.addition() << endl;

break;

case 2:
cout << "\tSubtraction : "<<a<<" & " <<b <<" is " << cal.subtraction() << endl;
break;
case 3:
   cout << "\tMultiplication : "<<a<<" & " <<b <<" is " << cal.multiplication() << endl;
   break;

      case 4:
cout << "\tDivision  :"<<a<<" & " <<b <<" is " << cal.division() << endl;
break;
```

```
default:
cout << "\tInvalid input " << endl;
break;
}
line();
        cout << "\n\t dear user do you want another Calculation [y/n] ";
        cout<<"\t";
        cin >> another;
        if (another == 'y' || another == 'Y')
        {
            goto again1;
        }
        else
            {
                    cout<<"\t dear user do you want to end the Program [y/n] ";
                    cout<<"\t";
                    cin>>terminate;
                    if(terminate=='y'||terminate=='Y')
                    {
                            break;
                                    }
                                    else
                                    goto program;
                            }
            break;
    }
    case 2:
    {
```

```
int choice2;
float c, d;
cout << "\t\tEnter two Floating Number ";
cout<<"\n\t";
cin >> c >> d;
Calculator<float, float> cal2(c, d);
again2:
cout<<"\n\tenter choice"<<endl;
cout << "\t1 for Addition \n\t2 for Subtraction \n\t3 for Multiplication \n\t 4 for Division"
<< endl;
cout<<"\n\t";
cin >> choice2;
system("cls");
line();
switch (choice2)
{
case 1:
    cout << "\tAddition :" <<c <<" & "<<d<< " is " << cal2.addition() << endl;
    break;

case 2:
    cout << "\tSubtraction : " <<c <<" & "<<d<< " is " << cal2.subtraction() << endl;
    break;

case 3:
    cout << "\tMultiplication : " <<c <<" & "<<d<< " is " << cal2.multiplication() << endl;
    break;

case 4:
```

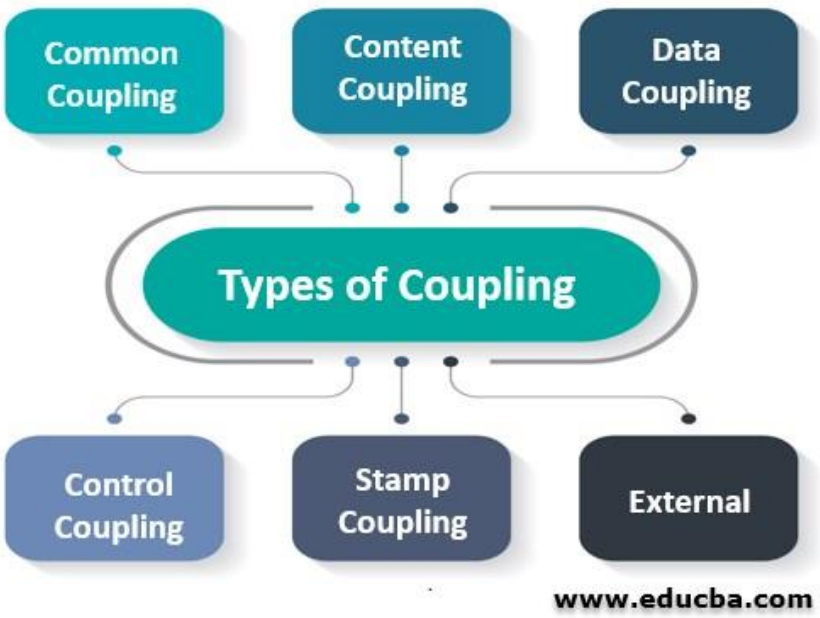
```
    cout << "\tDivision  :" <<c <<" & "<<d<< " is " << cal2.division() << endl;
    break;

default:
    cout << "\tInvalid input " << endl;
    break;
}
            line();
cout << "\t dear user do you want to  another Calculation [y/n] ";
cout<<"\t";
cin >> another;
if (another == 'y' || another == 'Y')
{
    goto again2;
}
else
    {
            cout<<"\t dear user do you want to Terminate Program [y/n] ";
            cout<<"\t";
            cin>>terminate;
            if(terminate=='y'||terminate=='Y')
            {
                    break;
                            }
                            else
                            goto program;
                    }
```

```
            break;
}

    case 3:
    {
        int choice3;
        float e, f;
        cout << "\tEnter a Floating Number and an Integer number ";
        cout<<"\n\t";
        cin >> e >> f;
        Calculator<float, int> cal3(e, f);
        again4:
        cout<<"\n\tEnter  Choice "<<endl;
        cout << "\n\t1) Addition \n\t2) Subtraction \n\t3) Multiplication \n\t4) Division" << endl;
        cout<<"\t";
        cin >> choice3;
        system("cls");
        line();
        switch (choice3)
        {
        case 1:
            cout << "\tAddition : "<<e<<" & " <<f <<" is " << cal3.addition() << endl;
            break;

        case 2:
            cout << "\tSubtraction :"<<e<<" & " <<f <<" is " << cal3.subtraction() << endl;
            break;
```

```
case 3:
    cout << "\tMultiplication : "<<e<<" & " <<f <<" is " << cal3.multiplication() << endl;
    break;

case 4:
    cout << "\tDivision  of "<<e<<" & " <<f <<" is " << cal3.division() << endl;
    break;

default:
    cout << "\tInvalid input " << endl;
    break;
}line;
            line();
cout << "\t dear user do you want another Calculation [y/n] ";
cout<<"\n\t";
cin >> another;
if (another == 'y' || another == 'Y')
{
    goto again4;
}
else
    {
            cout<<"\t dear user do you want to Terminate Program [y/n] ";
            cout<<"\t";
            cin>>terminate;
            if(terminate=='y'||terminate=='Y')
            {
                    break;
```

```
                    }
                    else
                    goto program;
                }
        break;
    }
    }

    return 0;
}
```

COUPLING



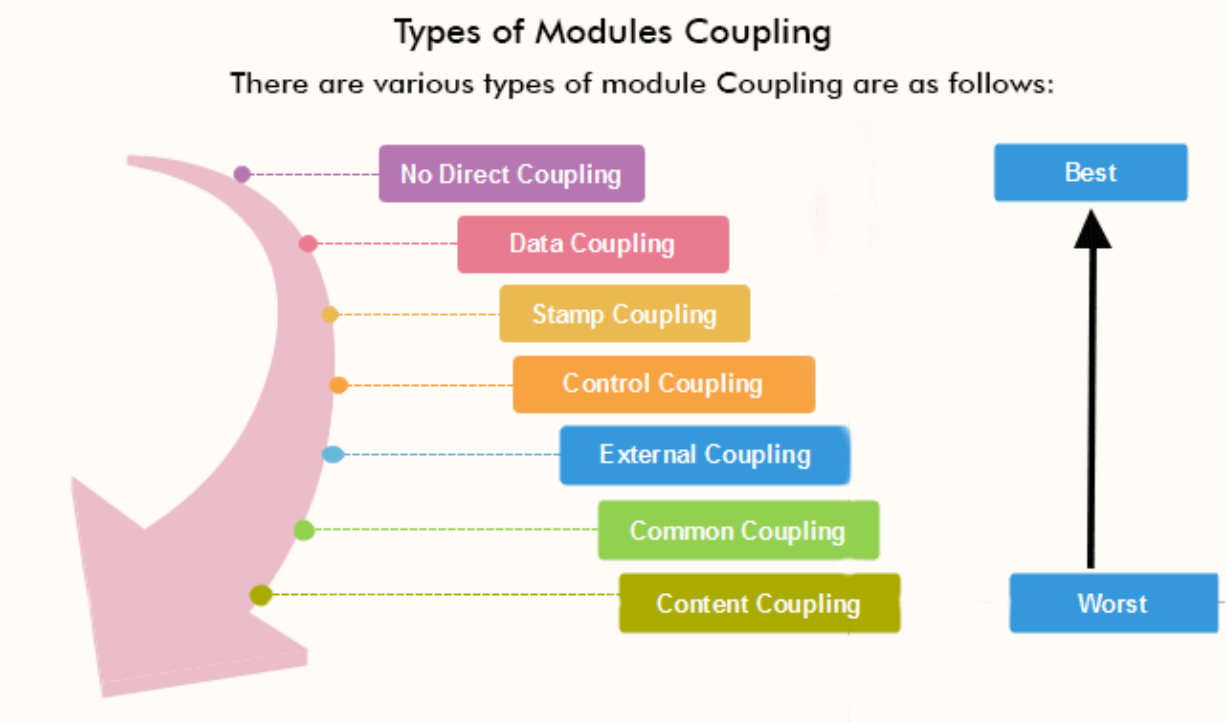
Content Coupling: This type of coupling occurs when one module directly accesses the internal details or data of another module. In the given program, content coupling is observed in the main function, where it directly accesses the private member variables of the Calculator **class (num1 and num2).**

Control Coupling: Control coupling exists when one module controls the execution flow or behavior of another module. In the program, the main function controls the flow of the program using **switch-case statements to** determine the actions to be taken by the Calculator class.

Stamp Coupling: Stamp coupling occurs when multiple data items are passed between modules as a single data structure or object. In the program, **there is no specific example of stamp coupling.**

External Coupling: External coupling exists when modules rely on external entities, such as global variables or libraries. In the program, there is a **line** example of external coupling.

Common Coupling: Common coupling occurs when multiple modules access the same global data or share a common resource. In the program, specific example of common coupling line, line is used as global coupling.



The program uses goto statements for control flow, which can lead to higher coupling and make the code more difficult to understand and maintain.

Lab-09: Cohesion concept by code analysis
COHESION

Code

```
#include <iostream>
using namespace std;

template <class A, class B>
class Calculator
{
    A num1;
    B num2;

public:
    Calculator(A num1, B num2)
    {
this->num1 = num1;
this->num2 = num2;
    }
A addition()
    {
 return num1 + num2;
    }

 A subtraction()
    {
 return num1 - num2;
    }
```

```
A multiplication()
    {    return num1 * num2;
    }

  A division()
    {
   return num1 / num2;
    }
};
void line()
{
            cout<<endl<<string(120,'_')<<endl;
}

int main()
{
    int option;
    char another,terminate;
    program:
    cout<<"\n\n\tenter choice "<<endl;
    cout << "\n\t 1 for Integers Calculation \n\t 2 for  Floating Calculations 3 for Integer &
Floating" << endl;
    cout<<"\t";
    cin >> option;
    switch (option)
    {
    case 1:
    {
int choice;
```

```
int a, b;
cout << "\tEnter two integers ";
cout<<"\t";
cin >> a >> b;
Calculator<int, int> cal(a, b);
again1:
cout<<"\n\n\tEnter your Choice"<<endl;
cout << "\n\t1) Addition \n\t2) Subtraction \n\t3) Multiplication \n\t4) Division" << endl;
cout<<"\t";
cin >> choice;
system("cls");
line();
switch (choice)
{
case 1:
cout << "\tAddition :"<<a<<" & " <<b <<" is " << cal.addition() << endl;

break;

case 2:
cout << "\tSubtraction : "<<a<<" & " <<b <<" is " << cal.subtraction() << endl;
break;
case 3:
   cout << "\tMultiplication : "<<a<<" & " <<b <<" is " << cal.multiplication() << endl;
   break;

      case 4:
cout << "\tDivision  :"<<a<<" & " <<b <<" is " << cal.division() << endl;
```

```
break;
default:
cout << "\tInvalid input " << endl;
break;
}
line();
        cout << "\n\t dear user do you want another Calculation [y/n] ";
        cout<<"\t";
        cin >> another;
        if (another == 'y' || another == 'Y')
        {
            goto again1;
        }
        else
            {
                    cout<<"\t dear user do you want to end the Program [y/n] ";
                    cout<<"\t";
                    cin>>terminate;
                    if(terminate=='y'||terminate=='Y')
                    {
                            break;
                                    }
                                    else
                                    goto program;
                            }
            break;
    }
    case 2:
```

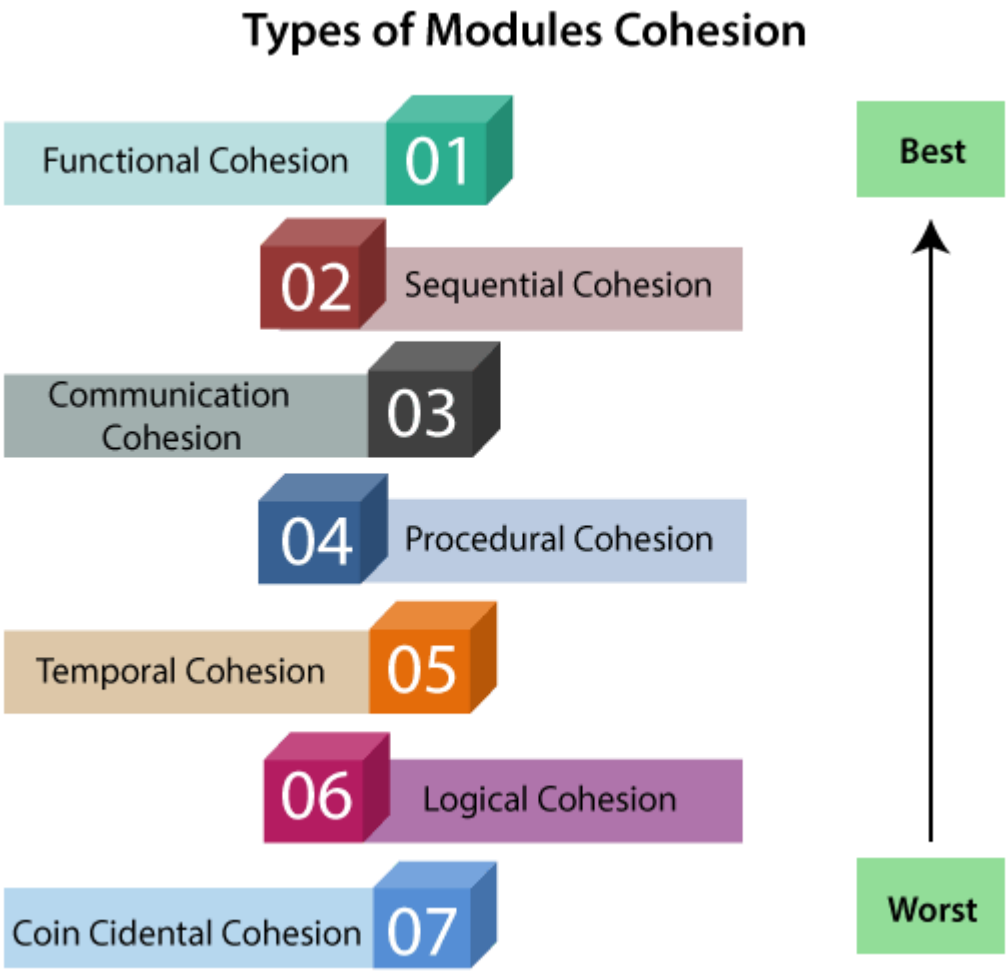
```
    {
        int choice2;
        float c, d;
        cout << "\t\tEnter two Floating Number ";
        cout<<"\n\t";
        cin >> c >> d;
        Calculator<float, float> cal2(c, d);
        again2:
        cout<<"\n\tenter choice"<<endl;
        cout << "\t1 for Addition \n\t2 for Subtraction \n\t3 for Multiplication \n\t 4 for Division"
<< endl;
        cout<<"\n\t";
        cin >> choice2;
        system("cls");
        line();
        switch (choice2)
        {
        case 1:
            cout << "\tAddition :" <<c <<" & "<<d<< " is " << cal2.addition() << endl;
            break;

        case 2:
            cout << "\tSubtraction : " <<c <<" & "<<d<< " is " << cal2.subtraction() << endl;
            break;

        case 3:
            cout << "\tMultiplication : " <<c <<" & "<<d<< " is " << cal2.multiplication() << endl;
            break;
```

```
case 4:
    cout << "\tDivision  :" <<c <<" & "<<d<< " is " << cal2.division() << endl;
    break;

default:
    cout << "\tInvalid input " << endl;
    break;
}
            line();
cout << "\t dear user do you want to  another Calculation [y/n] ";
cout<<"\t";
cin >> another;
if (another == 'y' || another == 'Y')
{
    goto again2;
}
else
    {
            cout<<"\t dear user do you want to Terminate Program [y/n] ";
            cout<<"\t";
            cin>>terminate;
            if(terminate=='y'||terminate=='Y')
            {
                    break;
                            }
                            else
                            goto program;
                    }
```

```
            break;
}

    case 3:
    {
        int choice3;
        float e, f;
        cout << "\tEnter a Floating Number and an Integer number ";
        cout<<"\n\t";
        cin >> e >> f;
        Calculator<float, int> cal3(e, f);
        again4:
        cout<<"\n\tEnter  Choice "<<endl;
        cout << "\n\t1) Addition \n\t2) Subtraction \n\t3) Multiplication \n\t4) Division" << endl;
        cout<<"\t";
        cin >> choice3;
        system("cls");
        line();
        switch (choice3)
        {
        case 1:
            cout << "\tAddition : "<<e<<" & " <<f <<" is " << cal3.addition() << endl;
            break;

        case 2:
            cout << "\tSubtraction :"<<e<<" & " <<f <<" is " << cal3.subtraction() << endl;
            break;
```

```
case 3:
    cout << "\tMultiplication : "<<e<<" & " <<f <<" is " << cal3.multiplication() << endl;
    break;

case 4:
    cout << "\tDivision  of "<<e<<" & " <<f <<" is " << cal3.division() << endl;
    break;

default:
    cout << "\tInvalid input " << endl;
    break;
}line;
            line();
cout << "\t dear user do you want another Calculation [y/n] ";
cout<<"\n\t";
cin >> another;
if (another == 'y' || another == 'Y')
{
    goto again4;
}
else
    {
            cout<<"\t dear user do you want to Terminate Program [y/n] ";
            cout<<"\t";
            cin>>terminate;
            if(terminate=='y'||terminate=='Y')
            {
```

```
                    break;
                        }
                        else
                        goto program;
                    }
            break;
    }
    }

    return 0;
}
```



Functional Cohesion: Functional cohesion is achieved when the code within a module is focused on performing a single, well-defined function. In the given program, the methods within the **Calculator class (addition, subtraction, multiplication, and division) exhibit functional cohesion as each method performs a specific mathematical operation.**

Sequential Cohesion: Sequential cohesion refers to the situation where the code within a module follows a sequential flow or order. In the program, **the main function follows a sequential order of execution, prompting the user for input, performing calculations, and displaying the results**.

Communicational Cohesion: Communicational cohesion is present when a module performs multiple functions, but they are all related to a specific data structure or input/output. In the program, **the Calculator class communicates the results of calculations back to the main function, which then displays the output to the user.**

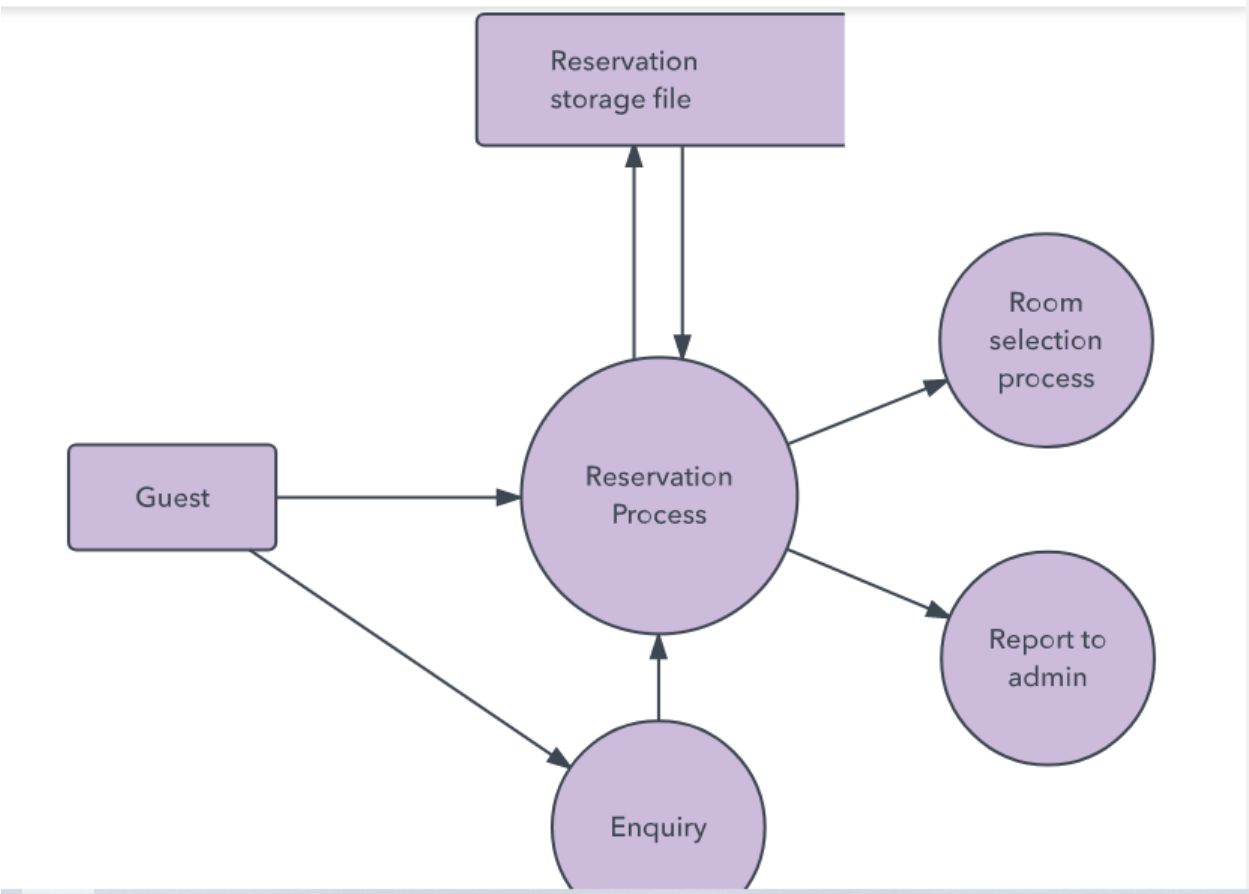
Temporal Cohesion: Temporal cohesion occurs when the code within a module is grouped together based on the timing or order of execution. In the program, the code related to each operation (addition, subtraction, multiplication, and division) is grouped together within **separate case statements in the switch-case blocks.**

Lab-10: Data Flow Diagram

DFD

- DFD (Data Flow Diagram) is a graphical representation of how data flows within a system or process. It consists of various symbols that represent entities and processes involved in the data flow. There are generally two levels of DFD: Level 0 and Level 1. Here's a detailed explanation of DFD symbols and their meanings for both levels:

- **Level 0 DFD Symbols and Meanings:**
- Process (Circle): Represents a specific function or action that transforms incoming data into outgoing data.
- External Entity (Rectangle): Represents an external agent, such as a user, system, or organization that interacts with the system but is not part of the system itself.
- Data Flow (Arrow): Represents the flow of data between processes, external entities, and data stores. It shows the direction of data movement.
- Data Store (Open-ended Rectangle): Represents a storage location for data, such as a database, file, or memory. It represents the persistent storage of data.



- **Level 1 DFD Symbols and Meanings:**
- Same symbols as Level 0 DFD.

- Decomposed Process (Circle with a plus sign): Represents a process that has been decomposed into more detailed sub-processes, allowing for a deeper level of analysis.
- Control Flow (Dashed Arrow): Represents the flow of control or sequence of activities within a process. It denotes the order in which activities occur.
- Data Flow (Arrow): Same as Level 0 DFD, representing the flow of data between processes, external entities, and data stores.
- Data Store (Open-ended Rectangle): Same as Level 0 DFD, representing a storage location for data.

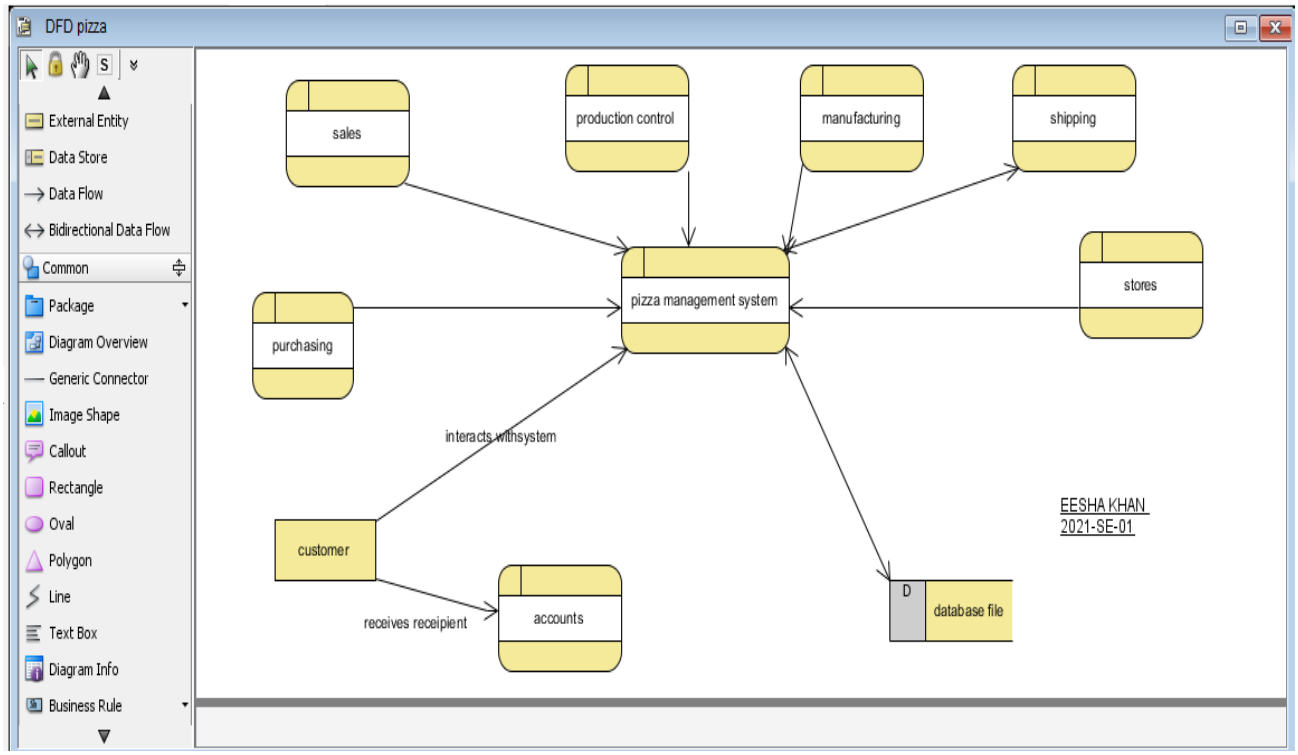
When to use

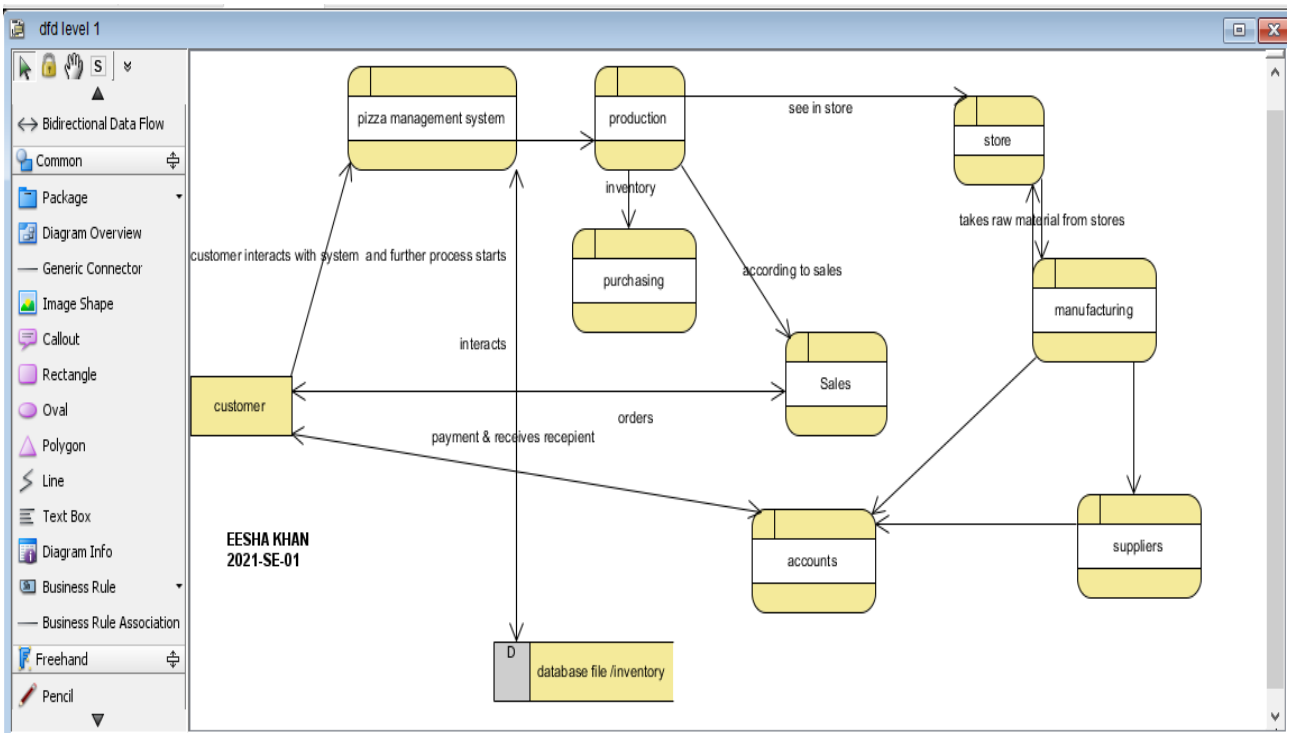
- When to use DFD and at which level depends on the complexity and scope of the system being analyzed. Here are some scenarios where DFDs can be effectively used:
- System Analysis and Design: DFDs are commonly used during the requirements gathering phase to visualize and analyze how data flows within a system, helping to identify functional requirements and potential bottlenecks.
- Process Improvement: DFDs can be used to analyze existing processes, identify inefficiencies, and suggest improvements by understanding the data flow and interactions between various entities.
- System Documentation: DFDs serve as effective documentation tools, helping stakeholders understand system processes, data flow, and interactions.
- Communication: DFDs provide a clear and concise way to communicate complex system concepts and requirements to both technical and non-technical stakeholders.

DFD rules and tips

- Each process should have at least one input and an output.
- Each data store should have at least one data flow in and one data flow out.
- Data stored in a system must go through a process.
- All processes in a DFD go to another process or a data store.

Implementations



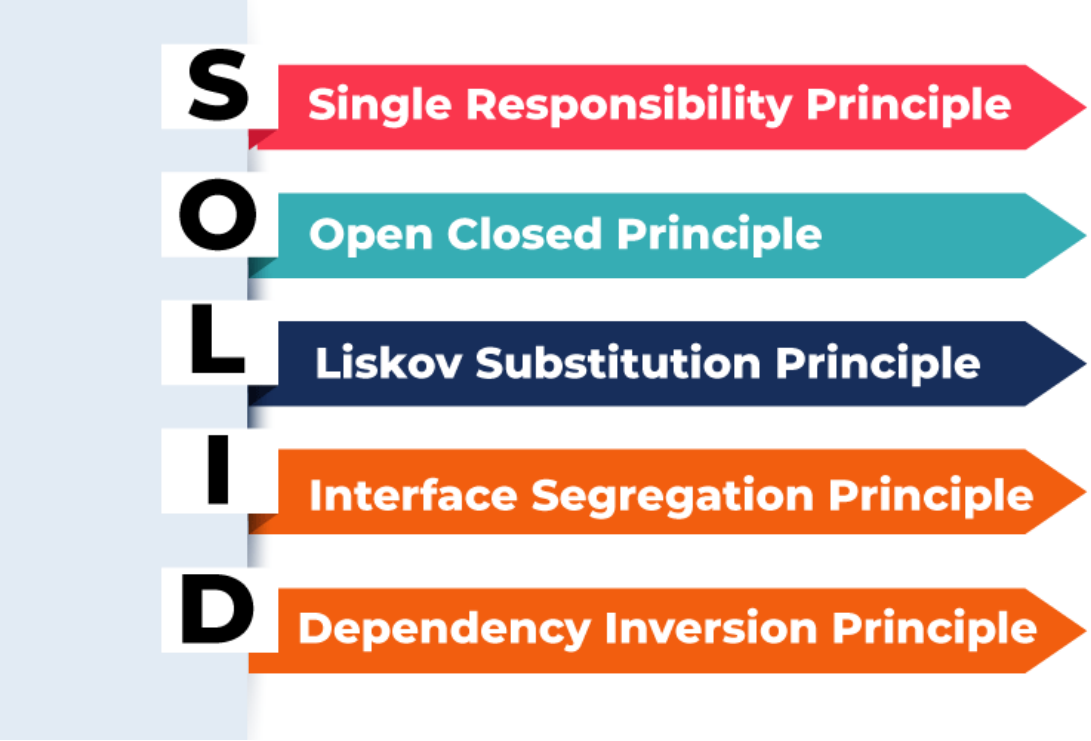


Lab-11: Design Principle 1
SOLID

SOLID is a popular set of design principles that are used in object-oriented software development. SOLID is an acronym that stands for five key design principles: single responsibility principle, open-closed principle, Liskov substitution principle, interface segregation principle, and dependency inversion principle. All five are commonly used by software engineers and provide some important benefits for developers.

Origin of SOLID

The SOLID principles were developed by Robert C. Martin in a 2000 essay, "Design Principles and Design Patterns," although the acronym was coined later by Michael Feathers. In his essay, Martin acknowledged that successful software will change and develop. As it changes, it becomes increasingly complex. Without good design principles, Martin warns that software becomes rigid, fragile, immobile, and viscous. The SOLID principles were developed to combat these problematic design patterns.

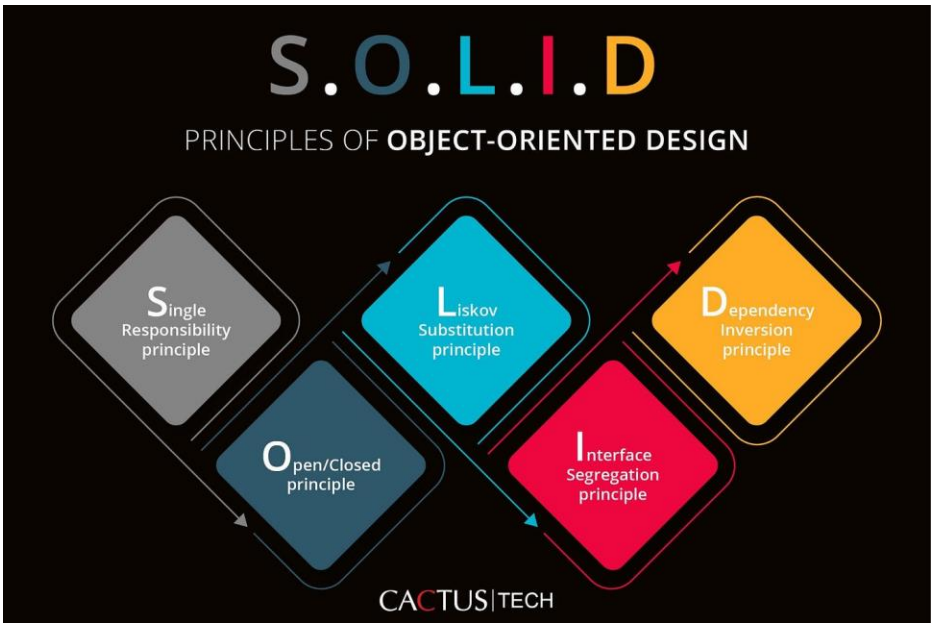


Goal of SOLID

The broad goal of the SOLID principles is to reduce dependencies so that engineers change one area of software without impacting others. Additionally, they're intended to make designs easier to understand, maintain, and extend. Ultimately, using these design principles makes it easier for software engineers to avoid issues and to build adaptive, effective, and agile software.

While the principles come with many benefits, following the principles generally leads to writing longer and more complex code. This means that it can extend the design process and make development a little more difficult. However, this extra time and effort is well worth it because it makes software so much easier to maintain, test, and extend.

Following these principles is not a cure-all and won't avoid design issues. That said, the principles have become popular because when followed correctly, they lead to better code for readability, maintainability, design patterns, and testability. In the current environment, all developers should know and utilize these principles.



1 Single Responsibility Principle

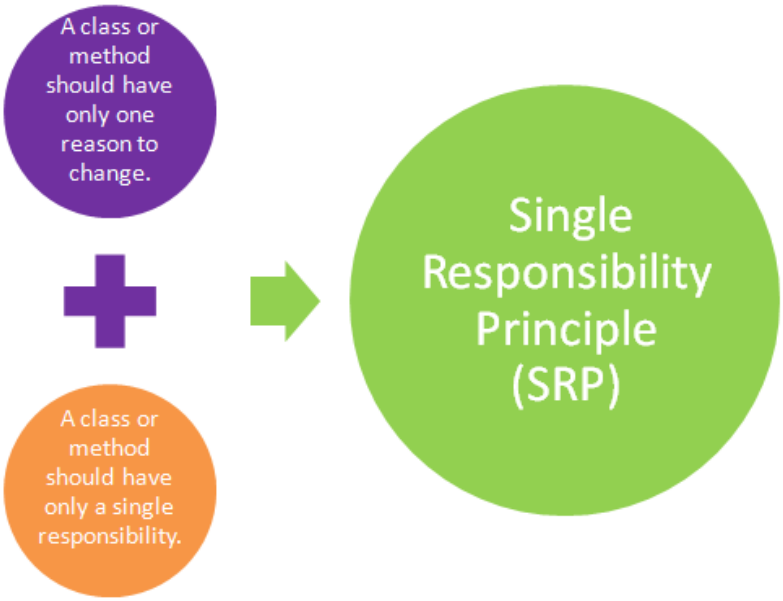
Robert Martin summarizes this principle well by mandating that, "a class should have one, and only one, reason to change**." Following this principle means that each class only does one thing and every class or module only has responsibility for one part of the software's functionality. More simply, each class should solve only one problem**.

Single responsibility principle is a relatively basic principle that most developers are already utilizing to build code. It can be applied to classes, software components, and microservices.

Utilizing this principle makes code easier to test and maintain, it makes software easier to implement, and it helps to avoid unanticipated side-effects of future changes.

To ensure that you're following this principle in development, consider using an automated check on build to limit the scope of classes. This check is not a foolproof way to make sure that

you’re following single responsibility principle, but it can be a good way to make sure that classes are not violating this principle.

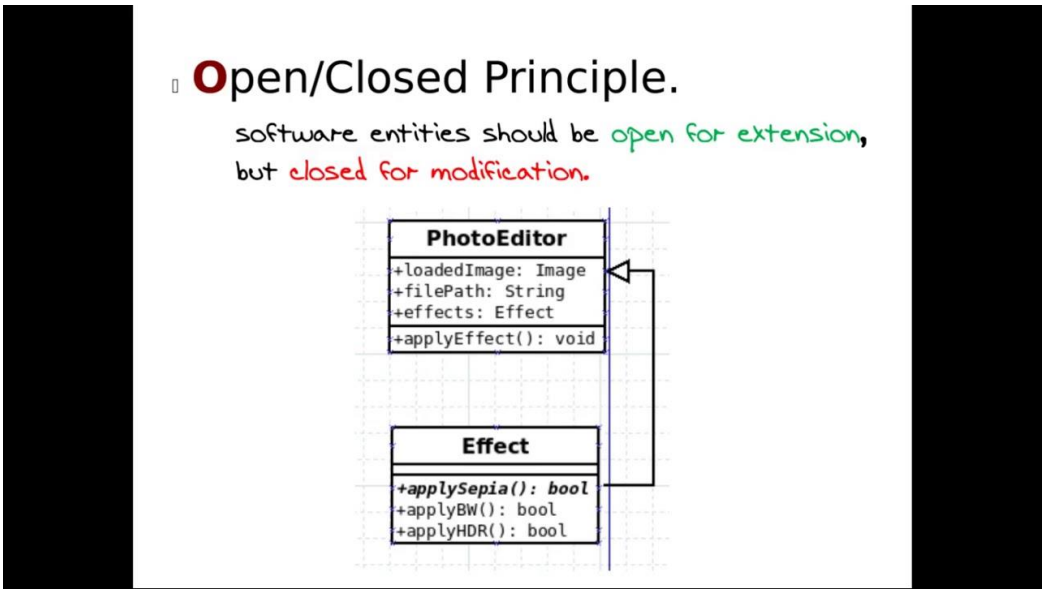


2 Open-Closed Principle

The idea of open-closed principle is that existing, well-tested classes will need to be modified when something needs to be added. Yet, changing classes can lead to problems or bugs. Instead of changing the class, you simply want to extend it. With that goal in mind, Martin summarizes this principle, “You should be able to extend a class’s behavior without modifying it.”

Open for extension, meaning that the class’s behavior can be extended; and

Closed for modification, meaning that the source code is set and cannot be changed.

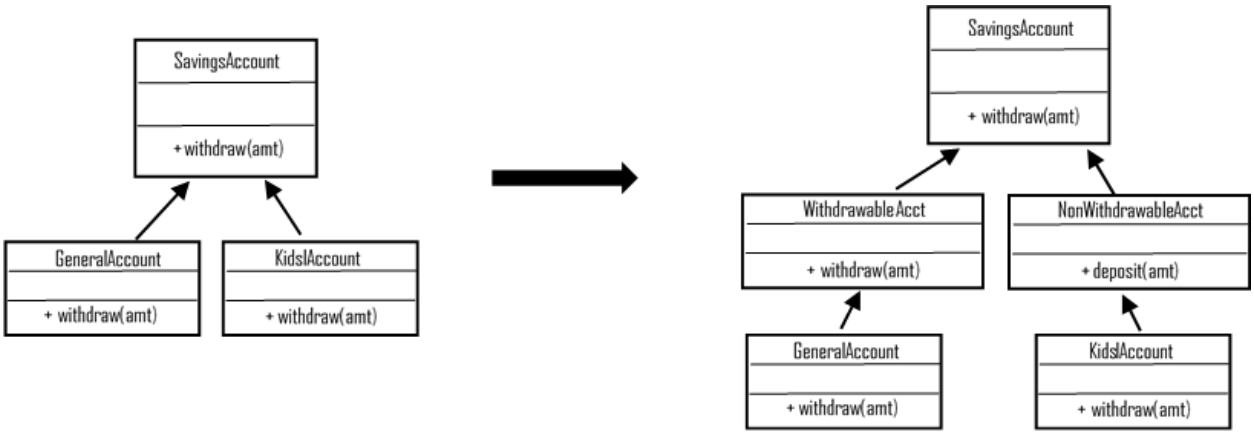


3 Liskov Substitution Principle

Of the five SOLID principles, the Liskov Substitution Principle is perhaps the most difficult one to understand. Broadly, this principle simply requires that every derived class should be substitutable for its parent class. **The principle is named for Barbara Liskov, who introduced this concept of behavioral subtyping in 1**987. Liskov herself explains the principle by saying:

What is wanted here is something like the following substitution property: if for each object O1 of type S there is an object O2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when O1 is substituted for O2 then S is a subtype of T.

While this can be a difficult principle to internalize, in a lot of ways it's simply an extension of open-closed principle, as it's a way of ensuring that derived classes extend the base class without changing behavior.



Implementations

```
#include <iostream>

class User {
private:
    Std::string name;
    Std::string email;
    int age;

public:
    User(std::string name, std::string email, int age) {
        this->name = name;
        this->email = email;
        this->age = age;
    }

    std::string getName() {
        return name;
    }

    std::string getEmail() {
        return email;
    }

    int getAge() {
        return age;
    }

    void sendEmail(std::string message) {
```

```
        std::cout << "Sending email to " << email << ": " << message << std::endl;
        // Some code to send an email
    }

    void printDetails () {
        std::cout << "Name: " << name << std::endl;
        std::cout << "Email: " << email << std::endl;
        std::cout << "Age: " << age << std::endl;
    }
};

int main() {
    User user("John Doe", "johndoe@example.com", 25);
    user.sendEmail("Hello, this is a test email.");
    user.printDetails();

    return 0;
}
```

Violation

Single Responsibility Principle (SRP)

The User class is responsible for both user data management (name, email, age) and sending emails. This violates SRP because the class has more than one reason to change. It would be better to separate the responsibilities into separate classes or modules.

Open/Closed Principle (OCP)

The User class is not closed for modification when new features need to be added. For example, if we wanted to add a new method for updating the user's information, we would need to modify the User class directly instead of extending it.

Liskov Substitution Principle (LSP)

Since there are no subclasses in this particular code snippet, LSP is not explicitly violated. However, if subclasses were introduced that inherit from the User class, they may not be able to substitute User objects in all cases without affecting the correctness of the program.

No violation

```
#include <iostream>
#include <string>

class User {
private:
    std::string name;
    std::string email;
    int age;

public:
    User(std::string name, std::string email, int age) {
        this->name = name;
        this->email = email;
        this->age = age;
    }

    std::string getName() {
        return name;
    }

    std::string getEmail() {
        return email;
    }
```

```
    int getAge() {
        return age;
    }
};

class EmailSender {
public:
    static void sendEmail(User user, std::string message) {
        std::cout << "Sending email to " << user.getEmail() << ": " << message << std::endl;
        // Some code to send an email
    }
};

class User Printer {
public:
    static void printDetails(User user) {
        std::cout << "Name: " << user.getName() << std::endl;
        std::cout << "Email: " << user.getEmail() << std::endl;
        std::cout << "Age: " << user.getAge() << std::endl;
    }
};

int main() {
    User user("John Doe", "johndoe@example.com", 25);
    EmailSender::sendEmail(user, "Hello, this is a test email.");
    UserPrinter::printDetails(user);

    return 0;
```

}

No violation

The User class now focuses solely on managing user data (name, email, age) and does not have any email sending or printing responsibilities. This adheres to the Single Responsibility Principle (SRP), as each class has a single responsibility.

The email sending functionality is moved to the EmailSender class, which has a static method sendEmail. This allows for better separation of concerns and ensures that the User class is not responsible for sending emails.

The user printing functionality is moved to the UserPrinter class, which also has a static method printDetails. This separates the responsibility of printing user details from the User class.

Lab-12: Design Principle II

1 Interface Segregation Principle

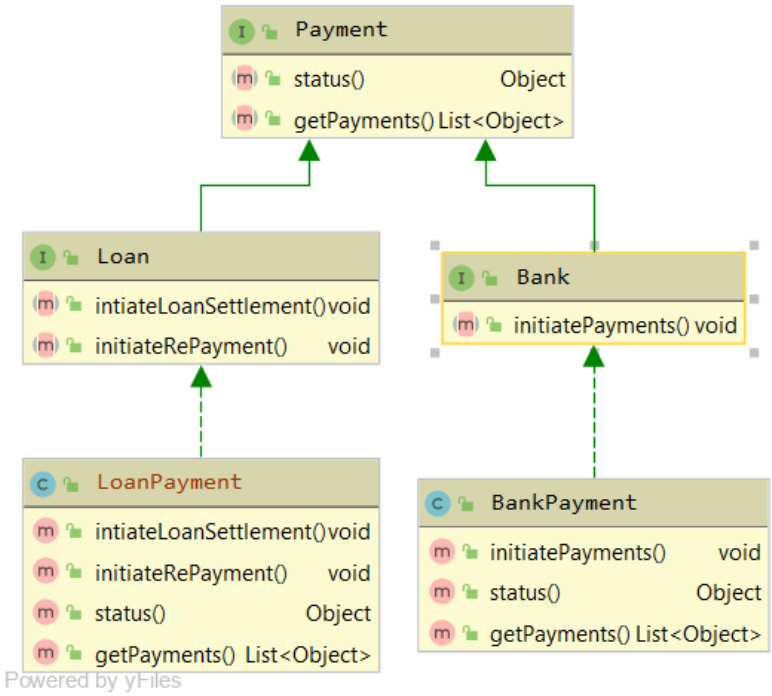
In the field of software engineering, the interface segregation principle (ISP) states that no code should be forced to depend on methods it does not use. ISP splits interfaces that are very large into smaller and more specific ones so that clients will only have to know about the methods that are of interest to them.

Example

One example of the Interface Segregation Principle violation: if an interface for a bank account has a withdrawal method, but you don't want to withdraw money, then the interface should be broken into an interface for depositing and one for withdrawing. This allows developers to use only the interfaces they need.

Use of interface segregation

Similar to the Single Responsibility Principle, the goal of the Interface Segregation Principle is to reduce the side effects and frequency of required changes by splitting the software into multiple, independent parts.



a

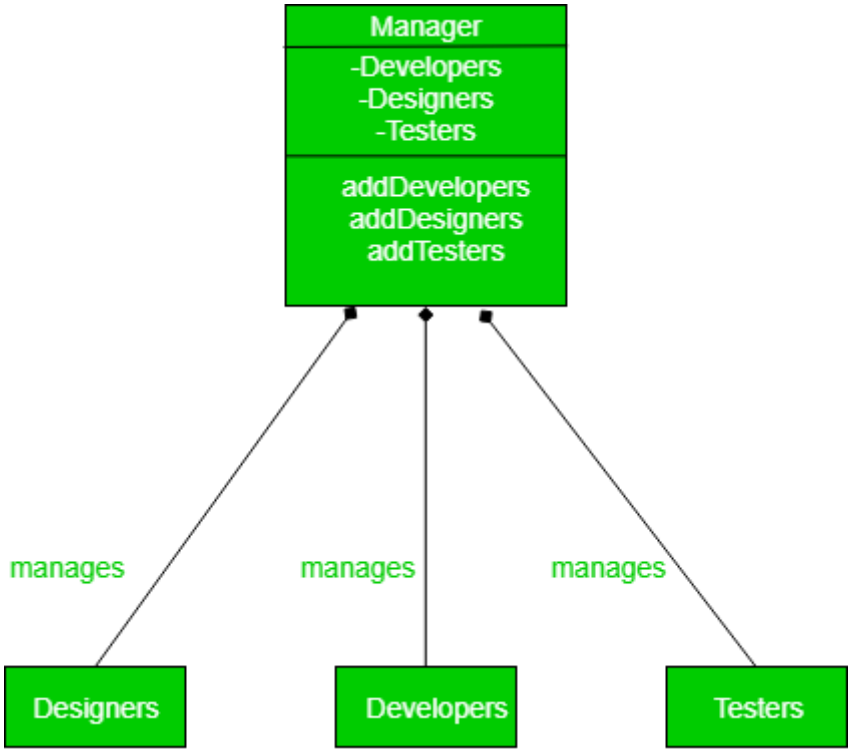
2 Dependency Inversion Principle

The dependency inversion principle is a design principle that states that high-level modules should depend on abstractions rather than concrete implementations. This helps decouple the high-level and low-level modules, making it easier to change the low-level ones without affecting the high-level ones.

Why is it called dependency inversion?

We call this dependency inversion because the high level policy no longer has a uses relationship with the low level policy but the low level policy now has an implements relationship on the abstraction. This implies that the high level policy and the abstraction reside on the same level.

Example



IMPLEMENTATIONS

```
#include <iostream>


// Interface for a BankAccount

class BankAccount {

public:
```

```
    virtual void deposit(double amount) = 0;
    virtual void withdraw(double amount) = 0;
    virtual double getBalance() = 0;
};

// Concrete implementation of BankAccount
class SavingsAccount : public BankAccount {
private:
    double balance;

public:
    SavingsAccount(double initialBalance) : balance(initialBalance) {}

    void deposit(double amount) override {
        balance += amount;
    }

    void withdraw(double amount) override {
        if (balance >= amount) {
            balance -= amount;
        } else {
            std::cout << "Insufficient funds!" << std::endl;
        }
    }

    double getBalance() override {
        return balance;
    }
};
```

```
// High-level module using the BankAccount interface
class ATM {
private:
    BankAccount* account;

public:
    ATM(BankAccount* acc) : account(acc) {}

    void performTransaction() {
        double amount = 100.0; // Assume a fixed amount for the transaction
        account->withdraw(amount);
        std::cout << "Remaining balance: " << account->getBalance() << std::endl;
    }
};

int main() {
    SavingsAccount savings(500.0);
    ATM atm(&savings);
    atm.performTransaction();

    return 0;
}
```

Violations

In this example, we have violated the Interface Segregation Principle because the Bank Account int**erface includes both deposit and withdrawal methods,** even though the ATM class only needs the withdrawal functionality. This forces all implementations of Bank Account , such as saving accounts to provide both deposit and withdrawal methods, even if they are not relevant for all account types.

Additionally, we have violated the Dependency Inversion Principle because the ATM class depends on the concrete implementation Savings Account instead of an abstraction. This makes it

difficult to switch to a different account type or introduce new account implementations without modifying the ATM class.

Corrections

```
#include <iostream>

// Interface for a BankAccount
class BankAccount {
public:
    virtual void deposit(double amount) = 0;
    virtual void withdraw(double amount) = 0;
    virtual double getBalance() = 0;
};

// Concrete implementation of BankAccount
class SavingsAccount : public BankAccount {
private:
    double balance;

public:
    SavingsAccount(double initialBalance) : balance(initialBalance) {}

    void deposit(double amount) override {
        balance += amount;
    }
```

```
    void withdraw(double amount) override {
        if (balance >= amount) {
            balance -= amount;
        } else {
            std::cout << "Insufficient funds!" << std::endl;
        }
    }

    double getBalance() override {
        return balance;
    }
};

// High-level module using the BankAccount interface
class ATM {
private:
    BankAccount* account;

public:
    ATM(BankAccount* acc) : account(acc) {}

    void performTransaction() {
        double amount = 100.0; // Assume a fixed amount for the transaction
        account->withdraw(amount);
        std::cout << "Remaining balance: " << account->getBalance() << std::endl;
    }
};
```

```
int main() {
    SavingsAccount savings(500.0);
    ATM atm(&savings);
    atm.performTransaction();

    return 0;
}
```

Lab-13: Design Patterns I

Singleton design pattern

The Singleton design pattern is a creational design pattern that restricts the instantiation of a class to a single object. It ensures that only one instance of a class exists throughout the application and provides a global point of access to that instance.

Here's a description of the key components and characteristics of the Singleton design pattern:

Single Instance

The Singleton pattern guarantees that only one instance of a class is created. This instance is shared by all the clients requesting access to it.

Private Constructor

To prevent external instantiation, the Singleton class typically has a private constructor. This prevents other classes from creating instances of the Singleton class directly.

Static Instance

The Singleton class contains a static member variable that holds the single instance of the class. This instance is typically initialized when the class is first accessed.

Access Point

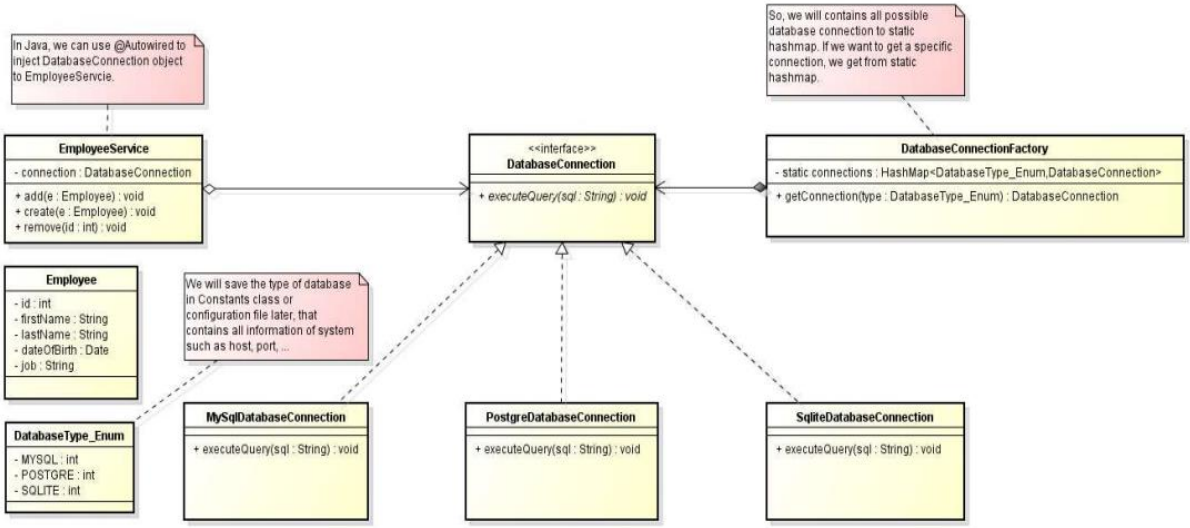
The Singleton class provides a global access point to the single instance, usually through a static method or property. Clients can use this access point to obtain the instance and perform operations on it.

Thread Safety

In a multi-threaded environment, special consideration must be given to ensure thread safety when implementing the Singleton pattern. Synchronization mechanisms like locks or double-checked locking can be used to prevent simultaneous creation of multiple instances.

The Singleton pattern is commonly used in scenarios where a single, shared resource needs to be accessed by multiple parts of an application. Examples include database connections, logging systems, configuration managers, and caches.

However, it's worth noting that the Singleton pattern has its drawbacks. It can introduce a global state and make testing and debugging more challenging. Overuse of the Singleton pattern can also lead to tight coupling and hinder flexibility and extensibility. Therefore, it's important to carefully consider whether the Singleton pattern is the appropriate choice for a given scenario.

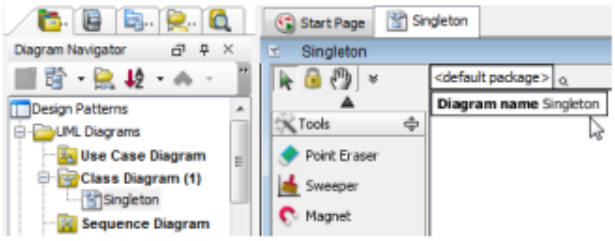


Steps

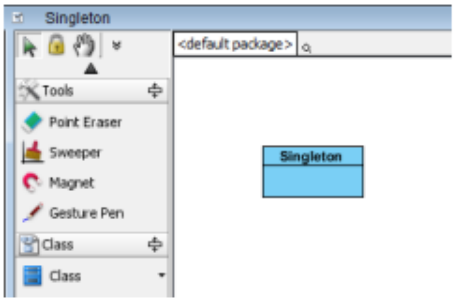
Modeling Design Pattern with Class Diagram

1. Create a new project *Design Patterns*.

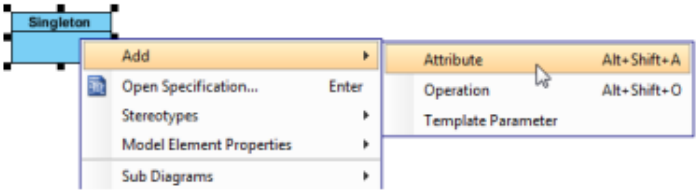
2. Create a class diagram *Singleton*.



3. Select **Class** from diagram toolbar. Click on the diagram to create a class. Name it as *Singleton*.

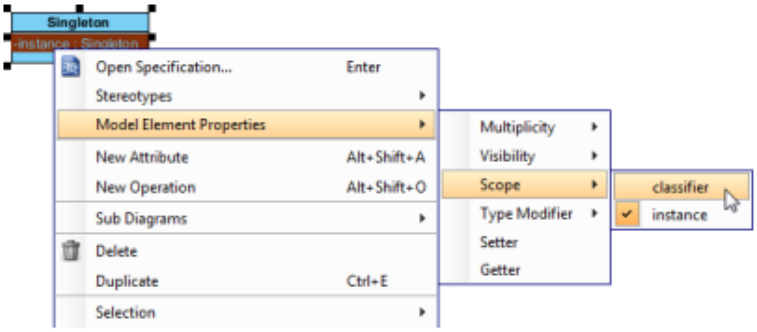


4. Right-click on the *Singleton* class and select **Add** > **Attribute** from the popup menu.

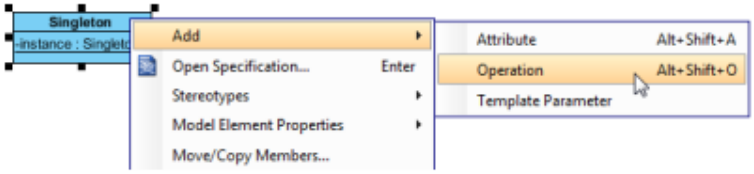


5. Name the attribute *instance*. Set its type as *Singleton*.

6. The attribute instance need to be static. Right-click on the attribut and select **Model Element Properties** > **Scope** > **Classifier** from the popup menu.



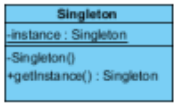
7. Create constructor for the *Singleton* class. Right-click on *Singleton* and select **Add** > **Operation** from the popup menu.



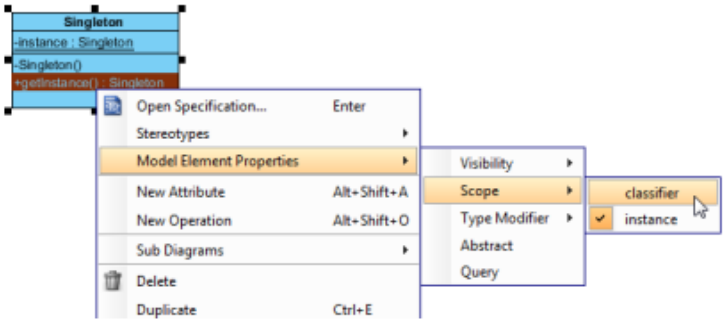
8. Name the operation *Singleton*, which follows the *Singleton* class's name. Change + to - in front of the operation name to indicate that this is a private constructor.

9. Right-click on *Singleton* and select **Add** > **Operation** from the popup menu.

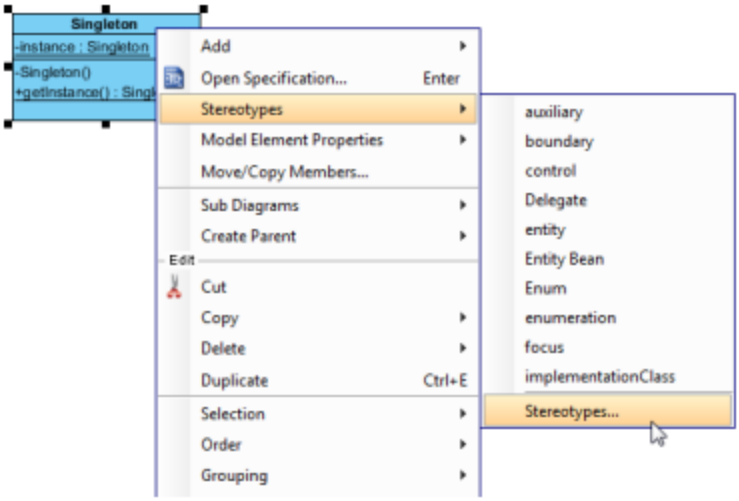
10. Name the operation *getInstance*, and make it return *Singleton*.



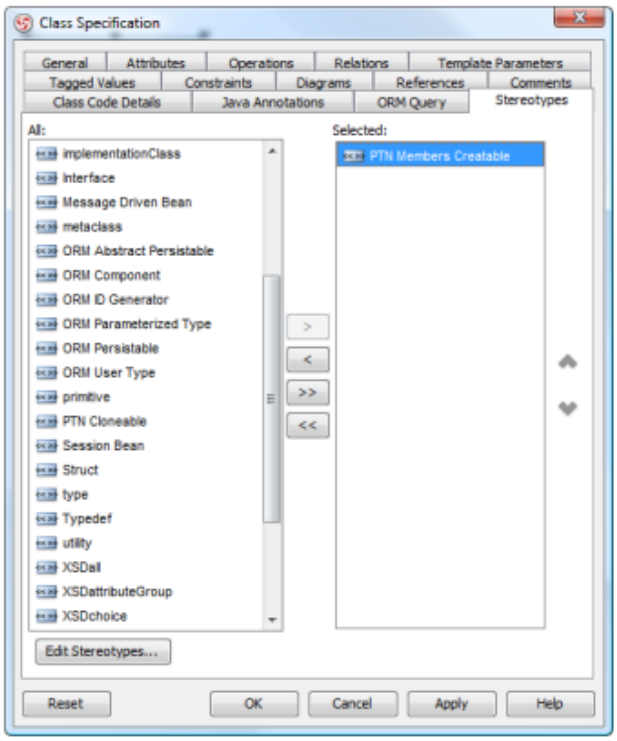
11. The operation *getInstance* need to be static. Right-click on the operation and select **Model Element Properties** > **Scope** > **Classifier** from the popup menu.



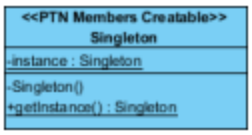
12. In practice, there may be operations for accessing data in the Singleton class. To represent this, stereotype the *Singleton* class as PTN Members Creatable. Right-click on the *Singleton* class and select **Stereotypes** > **Stereotypes...** from the popup menu.



13. In the class specification dialog box, select **PTN Members Creatable** and click > to assign it. Click **OK** to confirm.

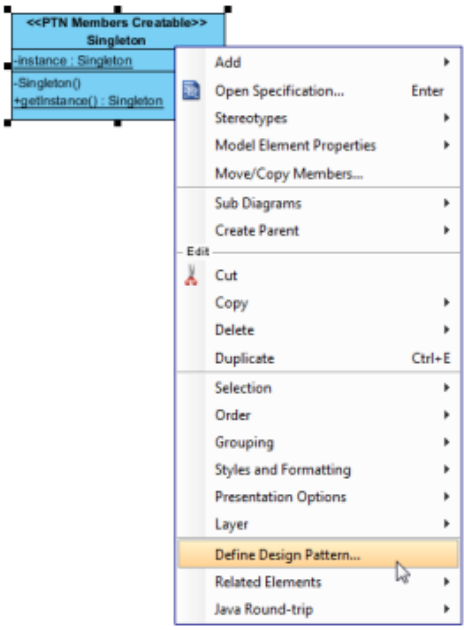


Up to now, the diagram should look like this:

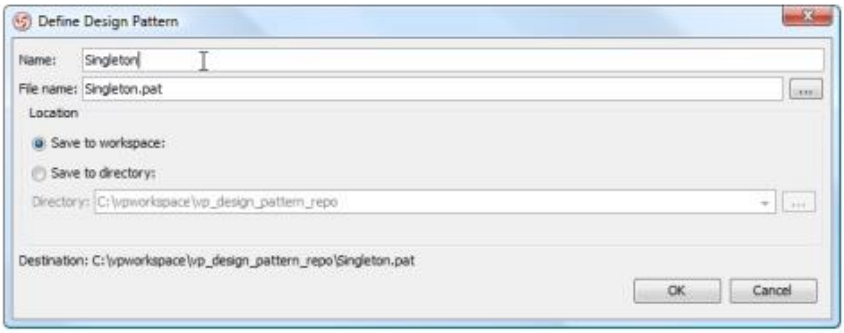


Defining Pattern

1. Right-click on the Singleton class and select **Define Design Pattern...** from the popup menu.



2. In the **Define Design Pattern** dialog box, specify the pattern name *Singleton*. Keep the file name as is. Click **OK** to proceed.

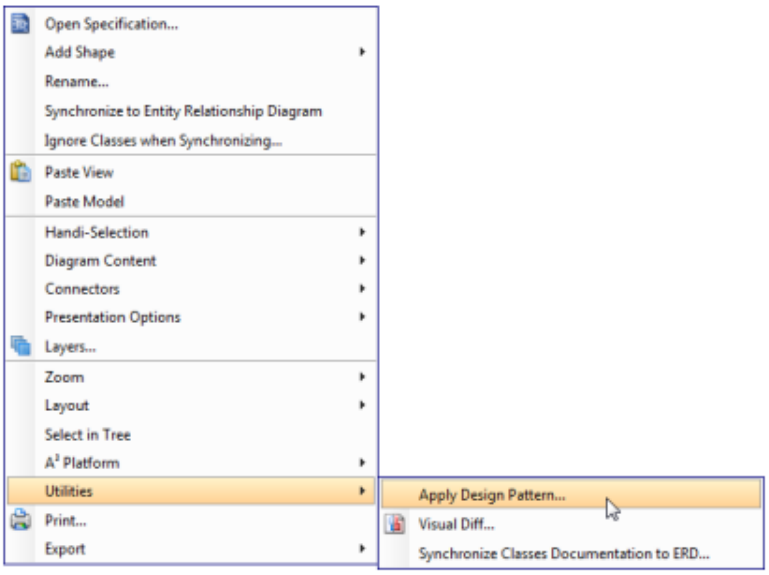


Applying Design Pattern on Class Diagram

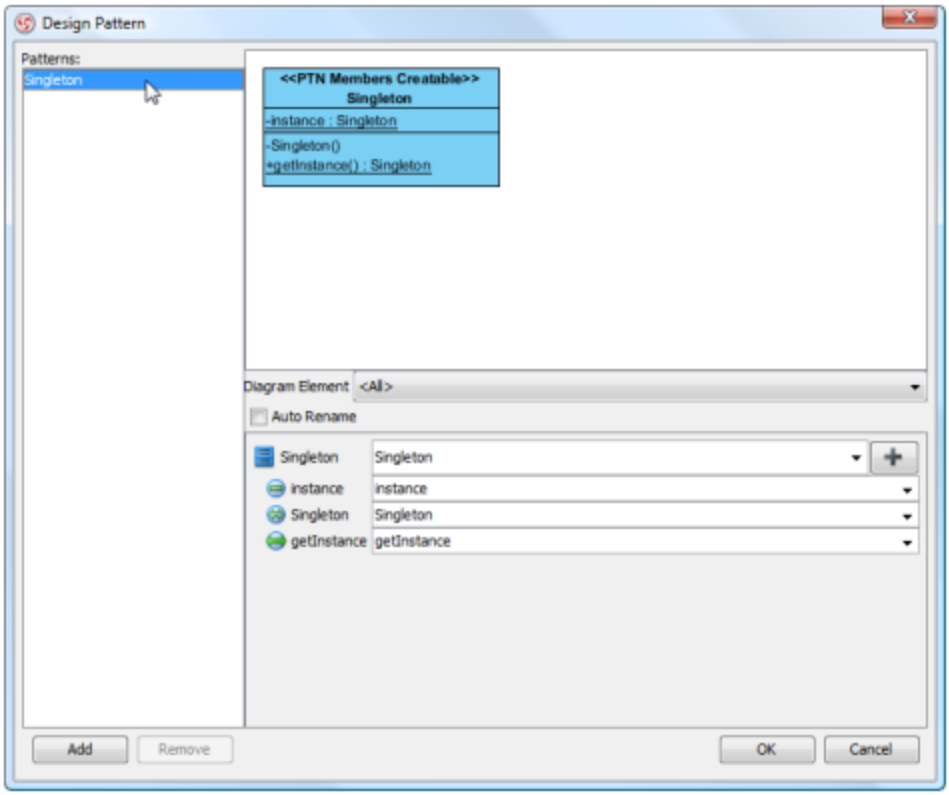
In this section, we are going to apply the singleton pattern in modeling a class registry.

1. Create a new project *Class Registry*.
2. Create a class diagram *The Registry*.

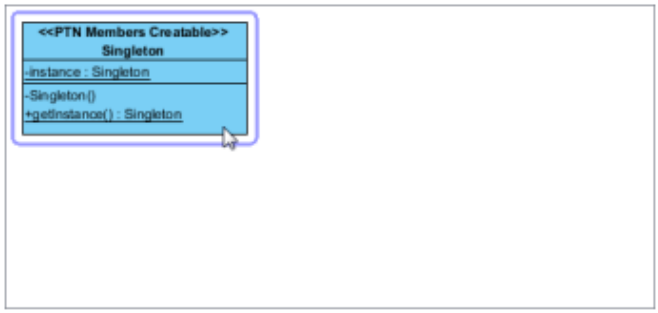
3. Right-click on the class diagram and select **Utilities** > **Apply Design Pattern...** from the popup menu.



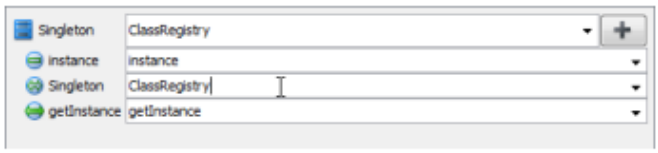
4. In the **Design Pattern** dialog box, select *Singleton* from the list of patterns.


Design Pattern
Patterns:
Singleton
<<PTN Members Creatable>>
Singleton
-instance : Singleton
-Singleton()
+getInstance() : Singleton
Diagram Element
<All>
Auto Rename
Singleton
Singleton
instance
instance
Singleton
Singleton
getInstance
getInstance
Add
Remove
OK
Cancel

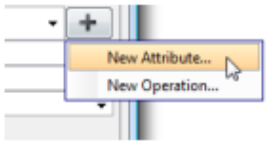
5. Click on *Singleton* in the ovewview.



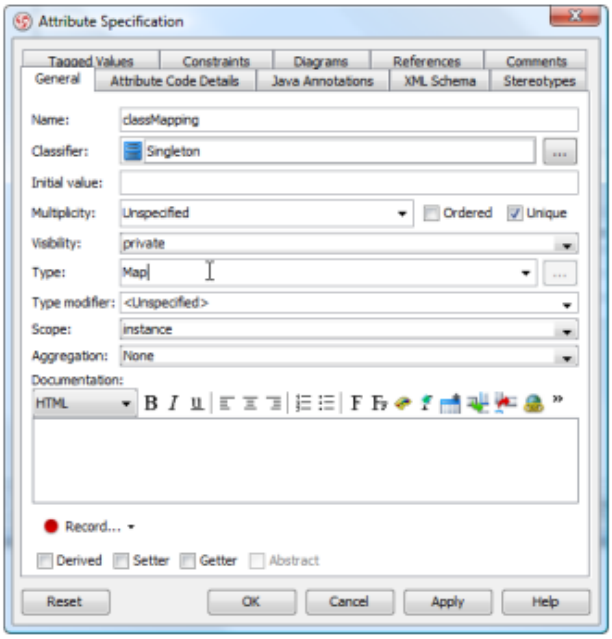
6. Rename the class *Singleton*, as well as the constructor to *ClassRegistry* at the bottom pane.



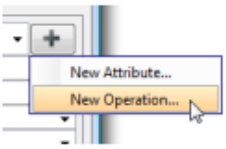
7. We need to add an attribute for holding the classes user register. Click on the + button and select **New Attribute...** from the popup menu.



8. In the **Attribute** Specification, enter *classMapping* as attribute name. Enter *Map* as type.

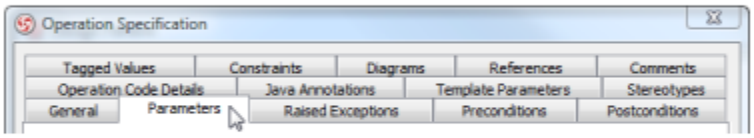


9. We need to add operations for registering class and retrieving class by type. Click on the + button and select **New Operation...** from the popup menu.



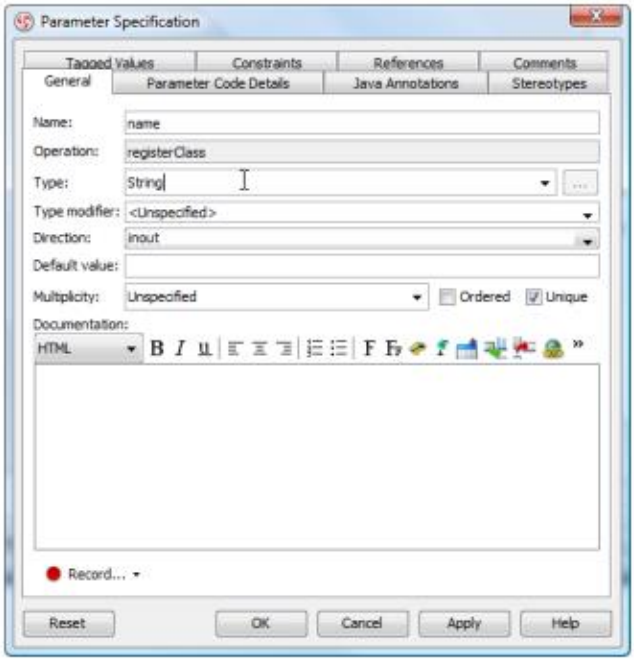
10. In the **Operation** Specification dialog box, enter *registerClass* as operation name.

11. Open the **Parameters** tab.

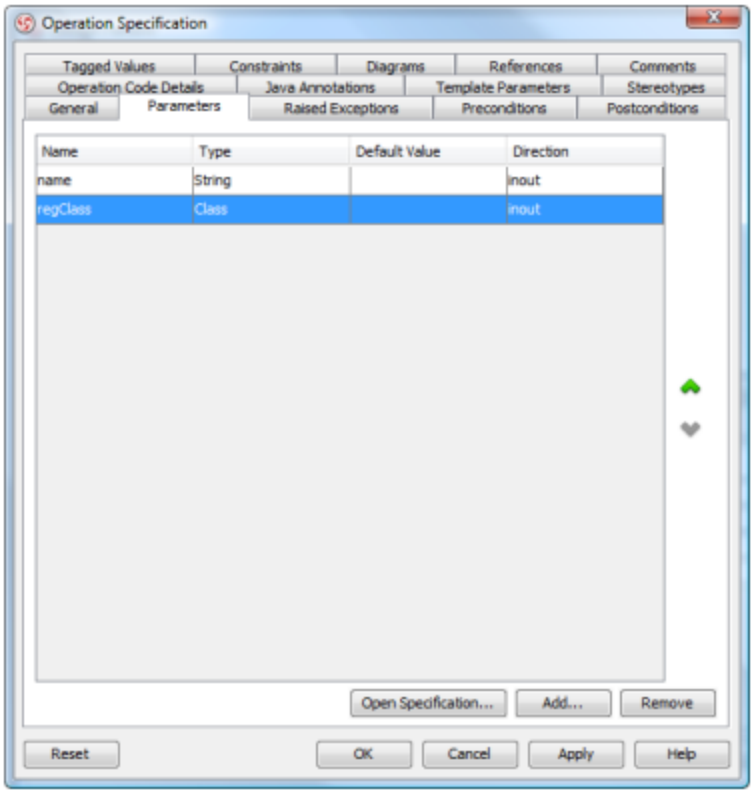


12. Click **Add...** at the bottom of specification dialog box.

13. In the **Parameter** Specification dialog box, enter *name* as parameter name and set *String* as type.

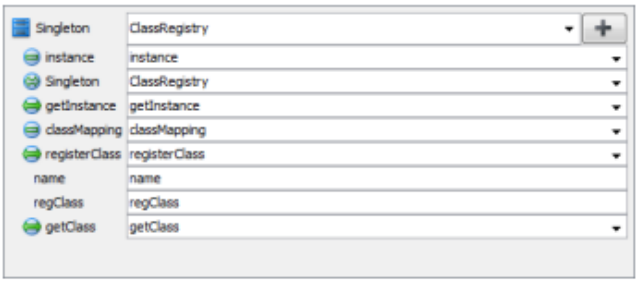


14. Repeat steps 12 and 13 to add parameter *regClass*, and set *Class* as type. Click **OK** to confirm.

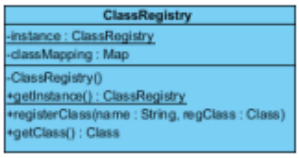

Operation Specification
Tagged Values
Constraints
Diagrams
References
Comments
Operation Code Details
Java Annotations
Template Parameters
Stereotypes
General
Parameters
Raised Exceptions
Preconditions
Postconditions
Name
Type
Default Value
Direction
name
String
inout
regClass
Class
inout
Open Specification...
Add...
Remove
Reset
OK
Cancel
Apply
Help

15. Click on the + button and select **New Operation...** from the popup menu.

16. In the **Operation Specification** dialog box, enter *getClass* as name, and set *Class* as return type.



17. Click **OK** to apply the pattern to diagram. This is the result:



Lab-14: Design Pattern II

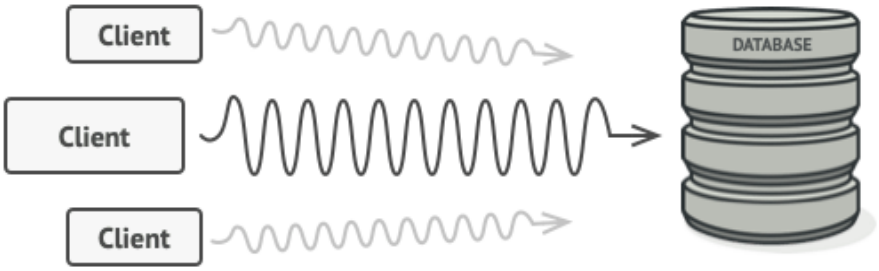
Proxy pattern

Proxy is a structural design pattern that lets you provide a substitute or placeholder for another object

- A proxy controls access to the original object
- allowing you to perform something either before or after the request gets through to the original object

Why would you want to control access to an object?

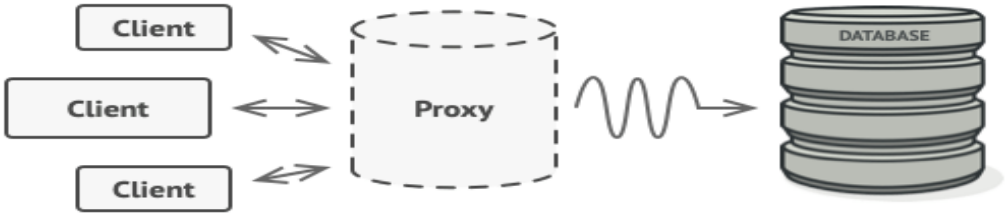
- Here is an example: you have a massive object that consumes a vast amount of system resources. You need it from time to time, but not always.



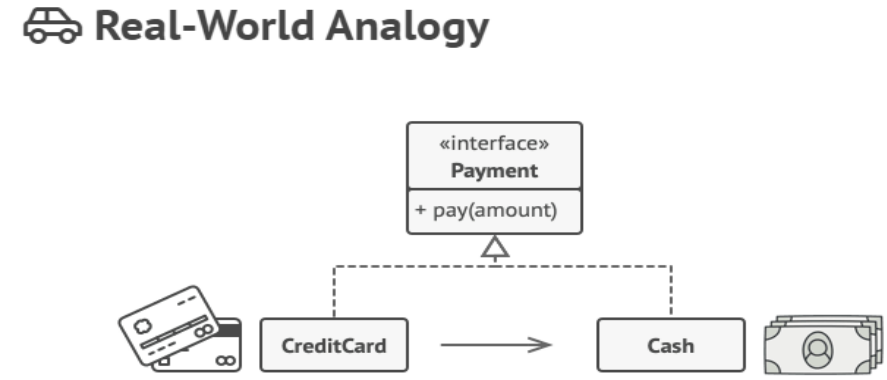
Database queries can be really slow.

Solution

- The Proxy pattern suggests that you create a new proxy class with the same interface as an original service object
- Then you update your app so that it passes the proxy object to all of the original object's clients
- Upon receiving a request from a client, the proxy creates a real service object and delegates all the work to it.

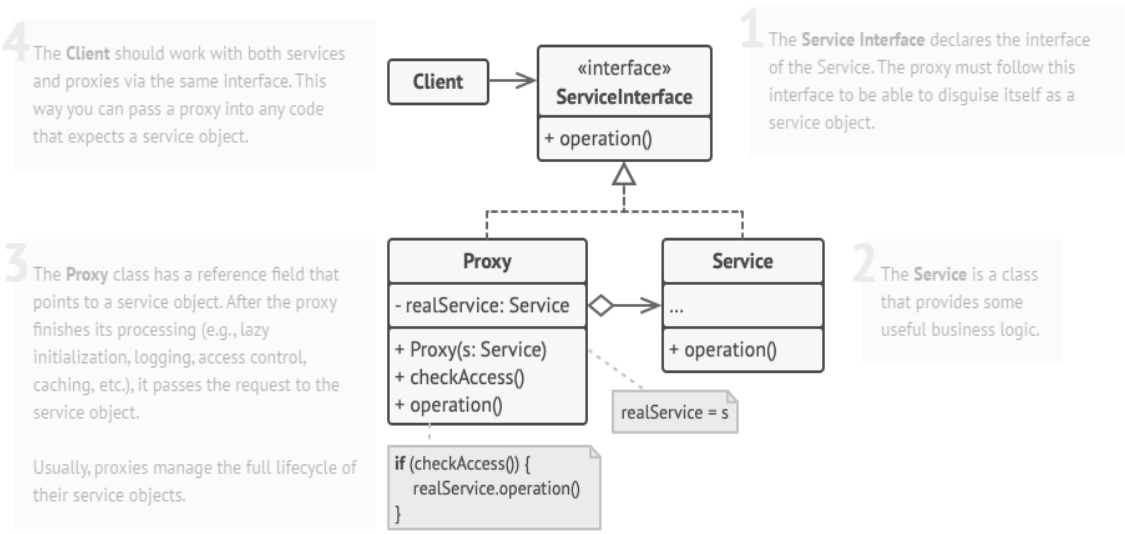


<u>Real world Analogy</u>



- A credit card is a proxy for a bank account, which is a proxy for a bundle of cash.

- Both implement the same interface: they can be used for making a payment.
- A consumer feels great because there's no need to carry loads of cash around.
- A shop owner is also happy since the income from a transaction gets added electronically to the shop's bank account without the risk of losing the deposit or getting robbed on the way to the bank.



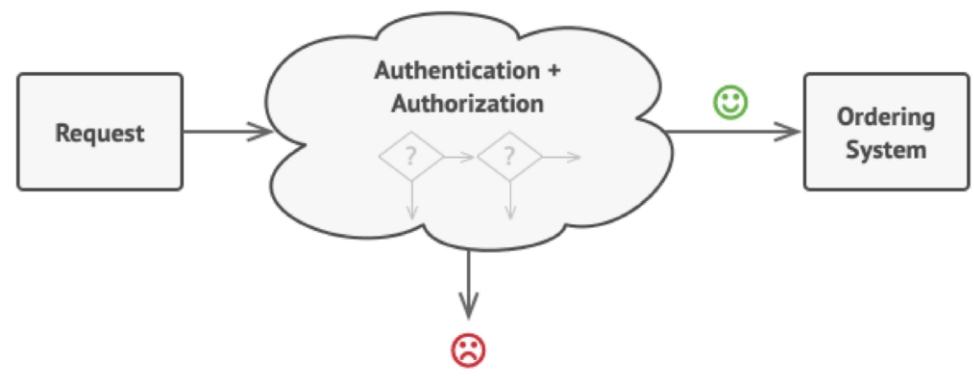
Template

```
interface Subject {
 void request();
} class Real Subject implements Subject {
public void request()
 {
 } }  class Proxy implements Subject {
private Real Subject real Subject;
public void request()
{
if (realSubject == null)
 { realSubject = new RealSubject();
} realSubject.request();
 } }
class Client {
private Subject subject;
public Client() { subject = new Proxy();
 } public void doSomething(
) { subject.request(); } }
```

Lab-15: Design Pattern III
Chain of Responsibility

Introduction to Chain of Responsibility Pattern: The Chain of Responsibility pattern is a behavioral design pattern that promotes the idea of passing a request along a chain of handlers. This chain consists of multiple objects, each responsible for either handling the request or passing it along to the next handler in the chain. This pattern decouples sender and receiver of a request, allowing multiple objects to potentially handle the request without the sender needing to know which object will eventually process it.

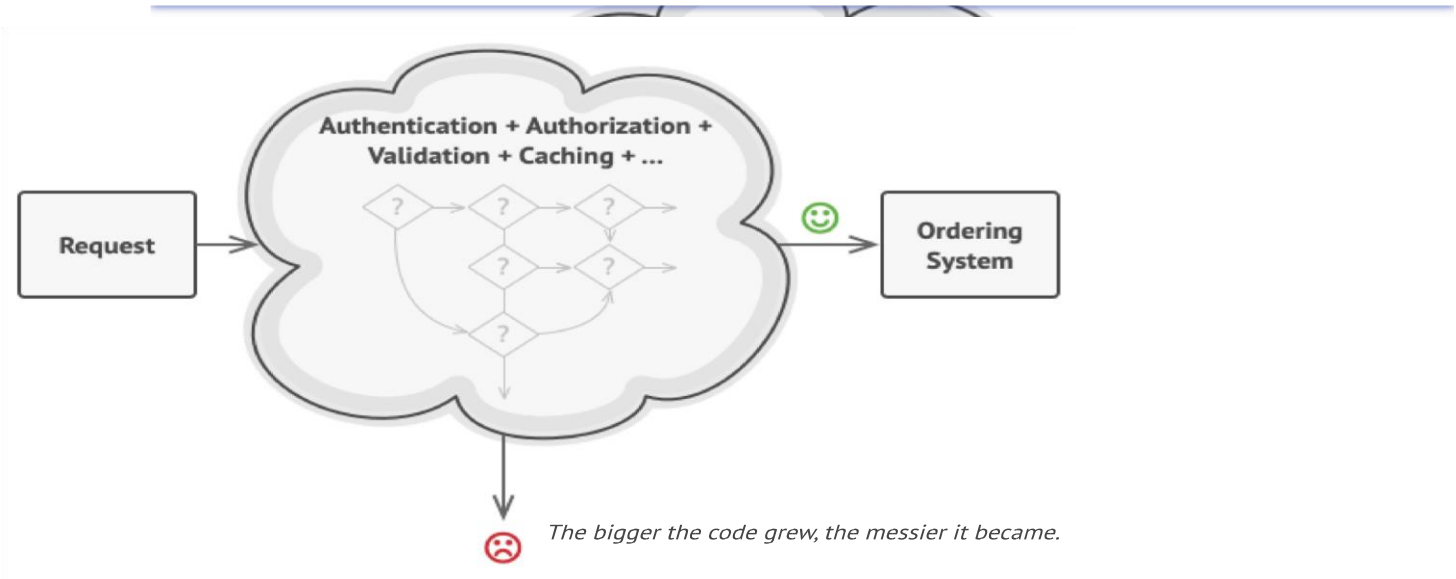
Imagine that you are working and restrict access to system so that only authenticated users can create the orders. Also administrative who have permission must have full access to full orders.



The request must pass a series of checks before the ordering system itself can handle it.

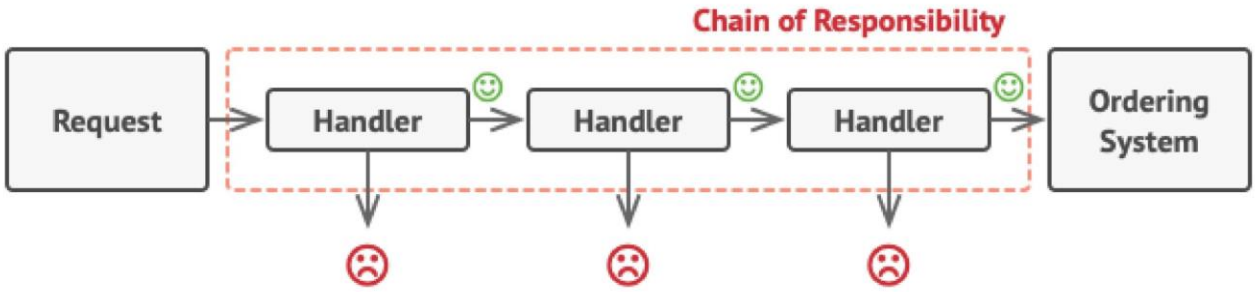
Authentication:

In the context of authentication and request processing, the Chain of Responsibility pattern can be utilized to implement a secure and flexible system. When a request is initiated, it enters the chain and passes through various authentication handlers. Each handler in the chain attempts to authenticate the request based on certain criteria, such as user roles, permissions, or credentials. If a handler successfully authenticates the request, it proceeds to the next phase of request processing. If not, the chain may **terminate the process or log the unauthorized attempt.**



Solution:

The Chain of Responsibility pattern provides a structured approach to handling authentication and request processing. It allows you to create a chain of authentication handlers, each specializing in a specific authentication method or level. For instance, you could have handlers for basic username/password authentication, two-factor authentication, token-based authentication, and more. If one handler fails to authenticate the request, it can pass the request to the next handler in the chain until a suitable handler is found or the request is denied.



Handlers are lined up one by one, forming a chain.

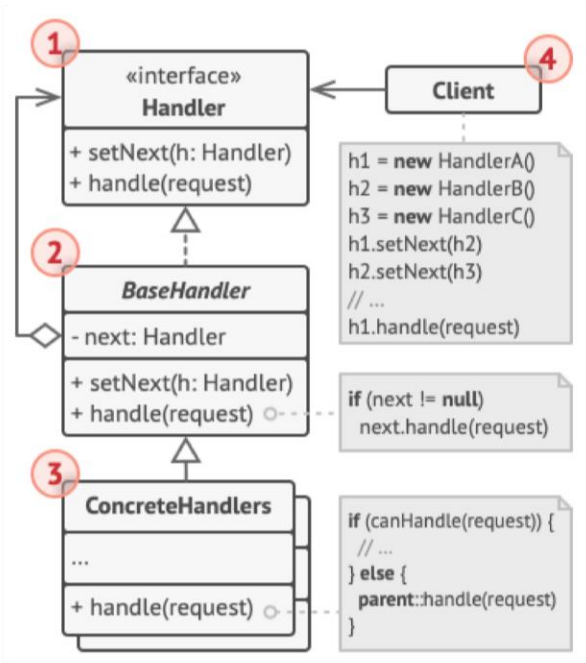
Structure

The main components of the Chain of Responsibility pattern include:

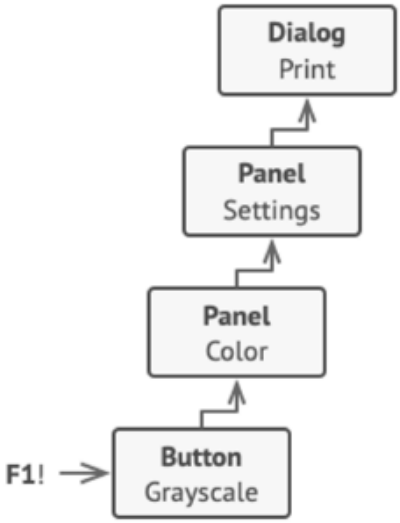
Handler: An abstract base class or interface that defines the common methods for handling requests and passing them to the next handler in the chain.

Concrete Handlers: Concrete subclasses that implement the actual authentication logic. Each concrete handler decides whether it can handle a request or not. If it can't, it delegates the request to the next handler.

Client: The entity that initiates the request and starts the chain. The client is unaware of the specific handlers in the chain and only interacts with the first handler.



Pseudo code




```
class Handler:
    constructor(successor = null):
        this.successor = successor

    method handleRequest(request):
        if this.successor is not null:
            this.successor.handleRequest(request)

class BasicAuthHandler extends Handler:
    method handleRequest(request):
        if request contains valid basic authentication credentials:
            // Authenticate the request
        else:
            super.handleRequest(request)

class TwoFactorAuthHandler extends Handler:
```

```
    method handleRequest(request):
        if request contains valid two-factor authentication code:
            // Authenticate the request
        else:
            super.handleRequest(request)

class TokenAuthHandler extends Handler:
    method handleRequest(request):
        if request contains valid token:
            // Authenticate the request
        else:
            // Request authentication failed
```

Pros and corns

Pros

Decoupling: Loose coupling between senders and receivers, enhancing flexibility.

Single Responsibility: Handlers have distinct responsibilities, improving code organization.

Extensibility: Easily add new handlers without altering existing code.

Dynamic Configuration: Chain can be configured at runtime for different scenarios.

Reusability: Handlers can be reused across various chains or projects.

Avoids Conditionals: Reduces complex conditionals for request handling.

Corns

Order Matters: Proper order of handlers is crucial for correct processing.

Performance Impact: Minor overhead due to multiple method calls.

Unhandled Requests: Poorly designed chains may leave requests unhandled.

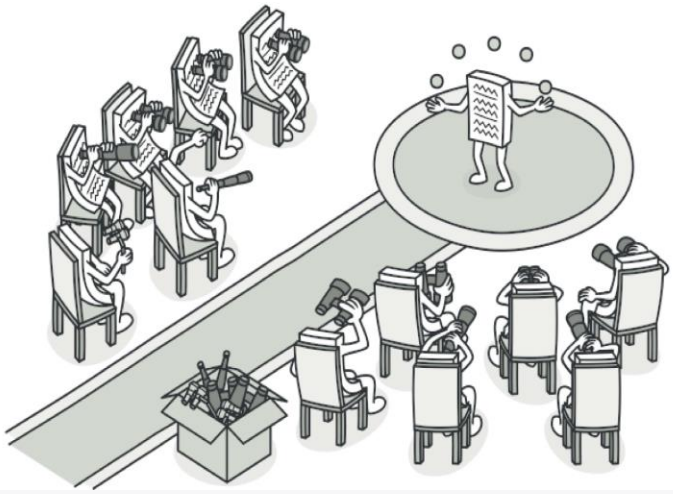
Complexity: Managing a large number of handlers can increase complexity.

Debugging Challenge: Tracking request paths through the chain can be intricate.

Inefficient Handling: Overlapping responsibilities might lead to inefficiencies.

Observer pattern

Introduction to Observer Pattern: The Observer pattern is a behavioral design pattern that establishes a one-to-many relationship between objects. When the state of one object (the



subject) changes, all its dependent objects (observers) are notified and updated automatically. This pattern is used to achieve loose coupling between objects, where changes in one object don't require changes in others.

Solution using Observer Pattern:

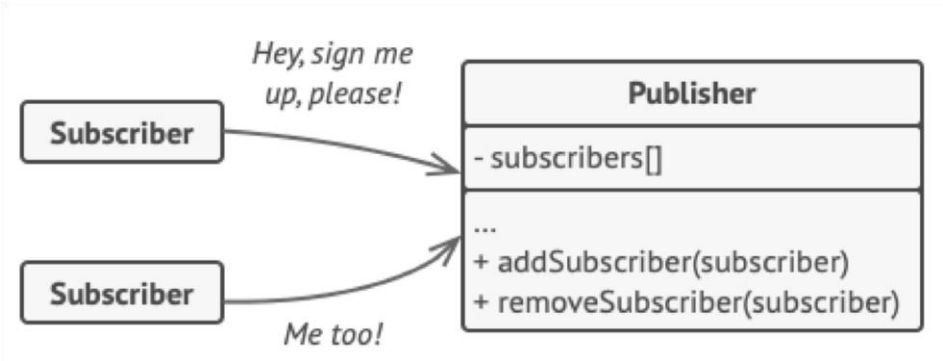
In situations where multiple objects need to react to changes in a central object, the Observer pattern offers an elegant solution. Rather than forcing objects to actively poll for changes, the Observer pattern enables automatic and efficient notification, allowing observers to take appropriate actions based on the new state of the subject.

Components of the Observer Pattern: Discuss the main components - Subject (Observable) and Observer (Subscriber).

Benefits of Decoupling: Explain how the Observer pattern promotes loose coupling and modularity.

Real-world Examples: Provide examples from various domains where the Observer pattern is useful.

Comparison with Other Patterns: Highlight the differences between the Observer pattern and other similar patterns like Publish-Subscribe.



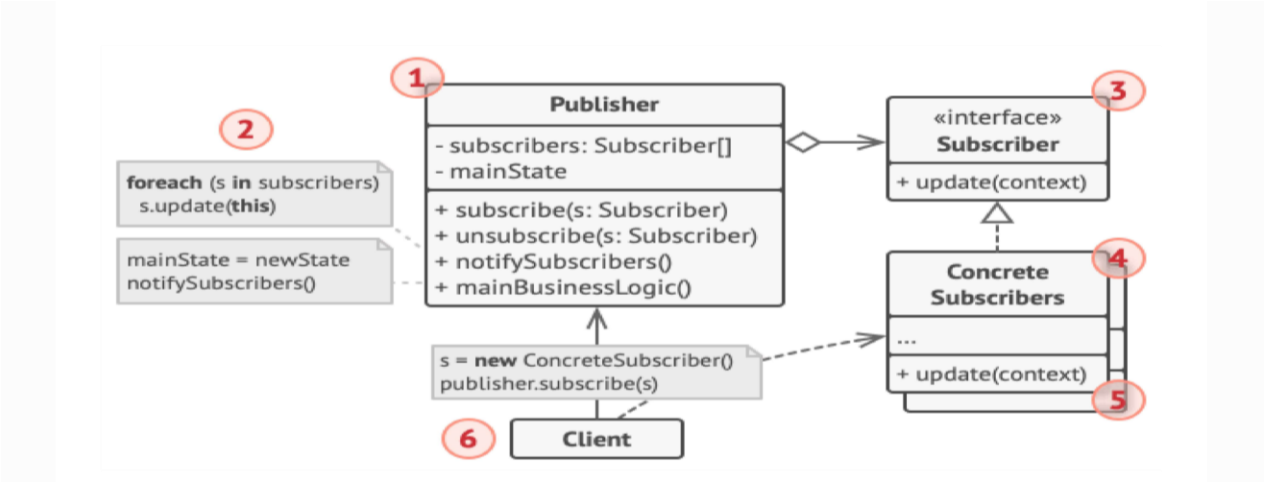
A subscription mechanism lets individual objects subscribe to event notifications.

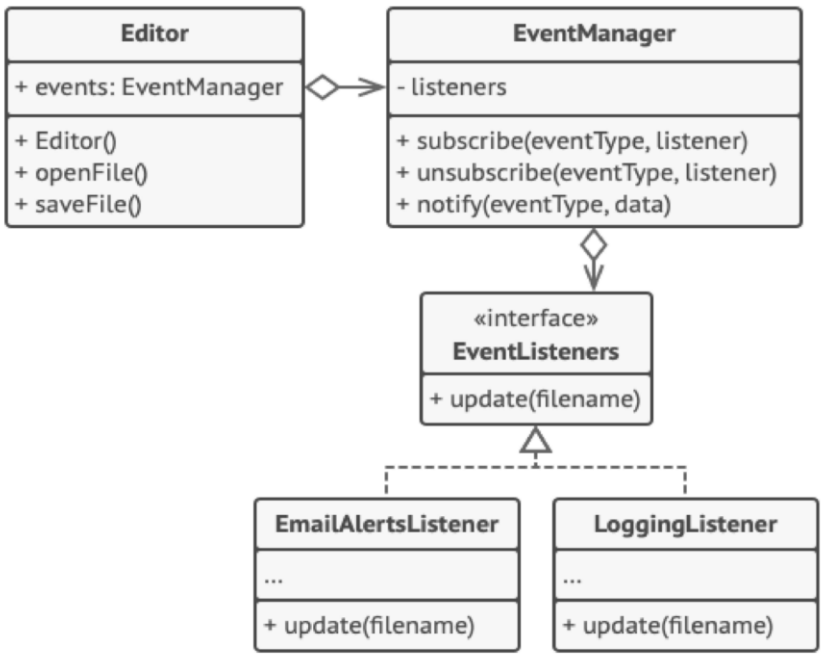
Pseudo code

```
class Subject:
    private observers = []

    method attach(observer):
        this.observers.add(observer)

    method detach(observer):
        this.observers.remove(observer)

    method notify():
        for observer in this.observers:
            observer.update()

class ConcreteSubject extends Subject:
    private state

    method setState(newState):
        this.state = newState
        this.notify()
```

```
    method getState():
        return this.state

class Observer:
    method update():
        // To be implemented by concrete observers

class ConcreteObserver implements Observer:
    private subject
    private observerState

    constructor(subject):
        this.subject = subject
        this.subject.attach(this)

    method update():
        this.observerState = this.subject.getState()
        // Perform necessary actions based on the new state
```

Structure





Notifying objects about events that happen to other objects.

Code

```
// The base publisher class includes subscription management
// code and notification methods.
class EventManager is
    private field listeners: hash map of event types and listeners

    method subscribe(eventType, listener) is
        listeners.add(eventType, listener)

    method unsubscribe(eventType, listener) is
        listeners.remove(eventType, listener)

    method notify(eventType, data) is
        foreach (listener in listeners.of(eventType)) do
            listener.update(data)

// The concrete publisher contains real business logic that's
// interesting for some subscribers. We could derive this class
// from the base publisher, but that isn't always possible in
// real life because the concrete publisher might already be a
// subclass. In this case, you can patch the subscription logic
// in with composition, as we did here.
class Editor is
    public field events: EventManager
    private field file: File

    constructor Editor() is
        events = new EventManager()
```

```
    constructor Editor() is
        events = new EventManager()

    // Methods of business logic can notify subscribers about
    // changes.
    method openFile(path) is
        this.file = new File(path)
        events.notify("open", file.name)

    method saveFile() is
        file.write()
```

```
// Here's the subscriber interface. If your programming language
// supports functional types, you can replace the whole
// subscriber hierarchy with a set of functions.
interface EventListener is
    method update(filename)

// Concrete subscribers react to updates issued by the publisher
// they are attached to.
class LoggingListener implements EventListener is
    private field log: File
    private field message: string

    constructor LoggingListener(log_filename, message) is
        this.log = new File(log_filename)
        this.message = message

    method update(filename) is
        log.write(replace('%s',filename,message))

class EmailAlertsListener implements EventListener is
    private field email: string
    private field message: string

    constructor EmailAlertsListener(email, message) is
        this.email = email
        this.message = message

    method update(filename) is
        system.email(email, replace('%s',filename,message))
```

```
// An application can configure publishers and subscribers at
// runtime.
class Application is
    method config() is
        editor = new Editor()

        logger = new LoggingListener(
            "/path/to/log.txt",
            "Someone has opened the file: %s")
        editor.events.subscribe("open", logger)

        emailAlerts = new EmailAlertsListener(
            "admin@example.com",
```

Pros of Observer Pattern:

Loose Coupling: Subjects and observers are decoupled, promoting better maintainability.

Dynamic Relationships: New observers can be added without modifying existing code.

Open-Closed Principle: Introducing new subjects or observers doesn't require modifying existing observers or subjects.

Real-time Updates: Observers receive updates immediately after a change in the subject.

Cons

Unintended Updates: Overuse of the pattern can lead to observers receiving updates they don't need.

Order of Notification: The order in which observers are notified might matter, which could be complex to manage.

Performance Impact: Notifying a large number of observers could introduce performance overhead.